

Groomsta Backend Structure Document

Document Information

Version: 1.0

Last Updated: December 2024

Prepared By: Senior Backend Engineering Team

Status: Ready for Implementation

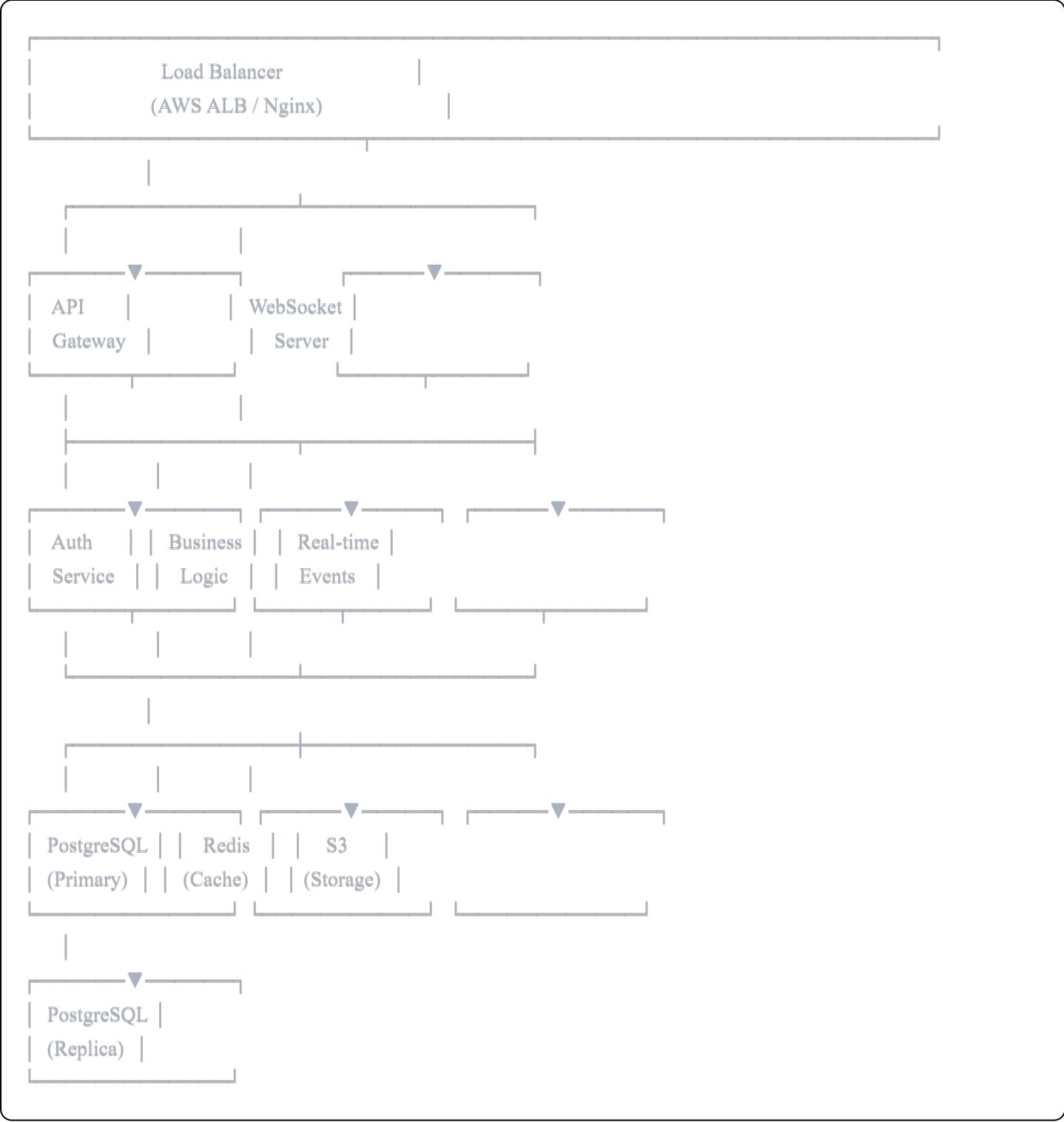
Table of Contents

1. [Architecture Overview](#)
 2. [Technology Stack](#)
 3. [Database Design](#)
 4. [API Structure](#)
 5. [Authentication & Authorization](#)
 6. [Real-time Communication](#)
 7. [Payment Integration](#)
 8. [Notification System](#)
 9. [Cron Jobs & Scheduled Tasks](#)
 10. [File Storage](#)
 11. [Caching Strategy](#)
 12. [Security Considerations](#)
 13. [Error Handling](#)
 14. [Logging & Monitoring](#)
 15. [Deployment Architecture](#)
-

1. Architecture Overview

1.1 System Architecture Pattern

Microservices-Ready Monolithic Architecture (Modular Monolith for MVP, easy migration to microservices)



1.2 Core Modules



└─ reviews/	# Ratings & Reviews
└─ referrals/	# Referral System
└─ memberships/	# Membership Plans
└─ admin/	# Admin Operations
└─ analytics/	# Metrics & Reporting
└─ common/	
└─ config/	# Configuration
└─ constants/	# Constants & Enums
└─ decorators/	# Custom Decorators
└─ dto/	# Data Transfer Objects
└─ guards/	# Route Guards
└─ interceptors/	# Response Interceptors
└─ middleware/	# Middleware
└─ utils/	# Utility Functions
└─ validators/	# Custom Validators
└─ database/	
└─ migrations/	# Database Migrations
└─ seeds/	# Seed Data
└─ entities/	# Database Entities
└─ integrations/	
└─ razorpay/	# Payment Gateway
└─ firebase/	# Push Notifications
└─ aws/	# AWS Services
└─ sms/	# SMS Gateway
└─ whatsapp/	# WhatsApp Business API
└─ email/	# Email Service

2. Technology Stack

2.1 Core Technologies

Component	Technology	Version	Justification
Runtime	Node.js	20.x LTS	Performance, async I/O, ecosystem
Framework	NestJS	10.x	Scalable, TypeScript, modular architecture
Language	TypeScript	5.x	Type safety, maintainability
Database	PostgreSQL	16.x	ACID compliance, JSON support, reliability
Cache	Redis	7.x	High-performance caching, pub/sub
ORM	TypeORM	0.3.x	TypeScript native, migrations, relations
Real-time	Socket.io	4.x	WebSocket, fallback support, rooms
API Documentation	Swagger	7.x	Auto-generated docs, testing interface

2.2 External Services

Service	Provider	Purpose
Payment Gateway	Razorpay	Online payments, refunds
File Storage	AWS S3 / Supabase	Document storage, images
SMS Gateway	MSG91 / Twilio	OTP, alerts
WhatsApp API	Gupshup / Twilio	Notifications
Email Service	SendGrid / AWS SES	Transactional emails
Push Notifications	Firebase Cloud Messaging	Real-time push
Maps API	Google Maps	Location, distance calculation

2.3 Development Tools

- **Logging:** Winston / Pino
- **Monitoring:** Sentry (errors), New Relic (APM)
- **API Testing:** Jest, Supertest
- **Code Quality:** ESLint, Prettier, Husky
- **Documentation:** Swagger/OpenAPI 3.0
- **Task Queue:** Bull (Redis-based)

3. Database Design

3.1 Database Schema

3.1.1 Users & Authentication

sql

-- Users Table (Customers)

```
CREATE TABLE users (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  phone VARCHAR(15) UNIQUE NOT NULL,  
  email VARCHAR(255) UNIQUE,  
  password_hash VARCHAR(255), -- For optional email login  
  full_name VARCHAR(255),  
  profile_image_url TEXT,  
  is_verified BOOLEAN DEFAULT FALSE,  
  is_active BOOLEAN DEFAULT TRUE,  
  last_login_at TIMESTAMP,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
  INDEX idx_users_phone (phone),  
  INDEX idx_users_email (email)  
);
```

-- OTP Verification

```
CREATE TABLE otp_verifications (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  phone VARCHAR(15) NOT NULL,  
  otp VARCHAR(6) NOT NULL,  
  purpose VARCHAR(50) NOT NULL, -- 'login', 'registration', 'verification'  
  expires_at TIMESTAMP NOT NULL,  
  is_used BOOLEAN DEFAULT FALSE,  
  attempts INT DEFAULT 0,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
  INDEX idx_otp_phone (phone),  
  INDEX idx_otp_expires (expires_at)  
);
```

-- Sessions

```
CREATE TABLE user_sessions (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,  
  partner_id UUID REFERENCES partners(id) ON DELETE CASCADE,  
  refresh_token VARCHAR(500) UNIQUE NOT NULL,  
  device_info JSONB, -- User agent, IP, device type  
  expires_at TIMESTAMP NOT NULL,  
  is_active BOOLEAN DEFAULT TRUE,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
  INDEX idx_sessions_user (user_id),  
  INDEX idx_sessions_partner (partner_id),
```

```
INDEX idx_sessions_token (refresh_token)
);
```

3.1.2 Partners

sql

-- Partners Table

```
CREATE TABLE partners (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  partner_type VARCHAR(50) NOT NULL, -- 'individual', 'salon'  
  phone VARCHAR(15) UNIQUE NOT NULL,  
  email VARCHAR(255) UNIQUE,  
  password_hash VARCHAR(255),  
  full_name VARCHAR(255) NOT NULL,  
  profile_image_url TEXT,  
  
  -- Professional Details  
  experience_years INT,  
  skills TEXT[],  
  bio TEXT,  
  
  -- Verification Status  
  verification_status VARCHAR(50) DEFAULT 'pending', -- 'pending', 'verified', 'rejected', 'suspended'  
  verified_at TIMESTAMP,  
  verified_by UUID REFERENCES admin_users(id),  
  rejection_reason TEXT,  
  
  -- Location  
  latitude DECIMAL(10, 8),  
  longitude DECIMAL(11, 8),  
  service_radius_km DECIMAL(5, 2) DEFAULT 10.0,  
  
  -- Availability  
  is_available BOOLEAN DEFAULT TRUE,  
  is_active BOOLEAN DEFAULT TRUE,  
  
  -- Ratings  
  average_rating DECIMAL(3, 2) DEFAULT 0.0,  
  total_ratings INT DEFAULT 0,  
  total_jobs_completed INT DEFAULT 0,  
  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
  INDEX idx_partners_phone (phone),  
  INDEX idx_partners_location (latitude, longitude),  
  INDEX idx_partners_status (verification_status, is_active)  
);
```

-- Partner Documents (KYC)

```
CREATE TABLE partner_documents (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```

partner_id UUID REFERENCES partners(id) ON DELETE CASCADE,
document_type VARCHAR(50) NOT NULL, -- 'aadhaar', 'pan', 'license', 'photo'
document_url TEXT NOT NULL,
document_number VARCHAR(100),
verification_status VARCHAR(50) DEFAULT 'pending',
verified_at TIMESTAMP,
verified_by UUID REFERENCES admin_users(id),
notes TEXT,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

INDEX idx_partner_docs_partner (partner_id),
INDEX idx_partner_docs_status (verification_status)
);

```

-- Partner Bank Details

```

CREATE TABLE partner_bank_details (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    partner_id UUID UNIQUE REFERENCES partners(id) ON DELETE CASCADE,
    bank_name VARCHAR(255) NOT NULL,
    account_holder_name VARCHAR(255) NOT NULL,
    account_number VARCHAR(50) NOT NULL,
    ifsc_code VARCHAR(20) NOT NULL,
    account_type VARCHAR(50), -- 'savings', 'current'
    is_verified BOOLEAN DEFAULT FALSE,
    verified_at TIMESTAMP,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    INDEX idx_bank_partner (partner_id)
);

```

-- Salon Details (if partner_type = 'salon')

```

CREATE TABLE salon_details (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    partner_id UUID UNIQUE REFERENCES partners(id) ON DELETE CASCADE,
    salon_name VARCHAR(255) NOT NULL,
    salon_address TEXT NOT NULL,
    pincode VARCHAR(10) NOT NULL,
    latitude DECIMAL(10, 8),
    longitude DECIMAL(11, 8),
    operating_hours JSONB, -- {"monday": {"open": "09:00", "close": "21:00"}, ...}
    amenities TEXT[],
    photos TEXT[],
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    INDEX idx_salon_partner (partner_id),

```



```

    INDEX idx_salon_location (latitude, longitude)
);

-- Partner Availability
CREATE TABLE partner_availability (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    partner_id UUID REFERENCES partners(id) ON DELETE CASCADE,
    day_of_week INT NOT NULL, -- 0=Sunday, 6=Saturday
    start_time TIME NOT NULL,
    end_time TIME NOT NULL,
    is_available BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    UNIQUE(partner_id, day_of_week),
    INDEX idx_availability_partner (partner_id)
);

-- Partner Blocked Dates
CREATE TABLE partner_blocked_dates (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    partner_id UUID REFERENCES partners(id) ON DELETE CASCADE,
    blocked_date DATE NOT NULL,
    reason TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    UNIQUE(partner_id, blocked_date),
    INDEX idx_blocked_partner_date (partner_id, blocked_date)
);

```

3.1.3 Services Catalog

```
sql
```

-- Service Categories

```
CREATE TABLE service_categories (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  name VARCHAR(255) NOT NULL,  
  slug VARCHAR(255) UNIQUE NOT NULL,  
  description TEXT,  
  icon_url TEXT,  
  display_order INT DEFAULT 0,  
  is_active BOOLEAN DEFAULT TRUE,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
  INDEX idx_categories_slug (slug),  
  INDEX idx_categories_order (display_order)  
);
```

-- Service Subcategories

```
CREATE TABLE service_subcategories (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  category_id UUID REFERENCES service_categories(id) ON DELETE CASCADE,  
  name VARCHAR(255) NOT NULL,  
  slug VARCHAR(255) NOT NULL,  
  description TEXT,  
  icon_url TEXT,  
  display_order INT DEFAULT 0,  
  is_active BOOLEAN DEFAULT TRUE,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
  UNIQUE(category_id, slug),  
  INDEX idx_subcategories_category (category_id),  
  INDEX idx_subcategories_slug (slug)  
);
```

-- Services

```
CREATE TABLE services (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  subcategory_id UUID REFERENCES service_subcategories(id) ON DELETE CASCADE,  
  name VARCHAR(255) NOT NULL,  
  slug VARCHAR(255) NOT NULL,  
  description TEXT,  
  long_description TEXT,
```

-- Pricing

```
base_price DECIMAL(10, 2) NOT NULL,  
partner_price_range_percent DECIMAL(5, 2) DEFAULT 20.0, -- ± percentage
```

-- Service Details

duration_minutes INT NOT NULL,
service_type VARCHAR(50) NOT NULL, -- 'home', 'salon', 'both'

-- Variants

has_gender_variants BOOLEAN DEFAULT FALSE,
has_location_variants BOOLEAN DEFAULT FALSE,

-- Media

image_url TEXT,
images TEXT[],

-- Status

is_active BOOLEAN DEFAULT TRUE,
display_order INT DEFAULT 0,

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

INDEX idx_services_subcategory (subcategory_id),
INDEX idx_services_slug (slug),
INDEX idx_services_type (service_type)

);

-- Service Variants

CREATE TABLE service_variants (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
service_id UUID REFERENCES services(id) ON DELETE CASCADE,
variant_type VARCHAR(50) NOT NULL, -- 'gender', 'location'
variant_value VARCHAR(100) NOT NULL, -- 'male', 'female', 'delhi', 'noida'
price_adjustment DECIMAL(10, 2) DEFAULT 0.0, -- Additional price
is_active BOOLEAN DEFAULT TRUE,

UNIQUE(service_id, variant_type, variant_value),
INDEX idx_variants_service (service_id)

);

-- Service Add-ons

CREATE TABLE service_addons (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
service_id UUID REFERENCES services(id) ON DELETE CASCADE,
name VARCHAR(255) NOT NULL,
description TEXT,
base_price DECIMAL(10, 2) NOT NULL,
duration_minutes INT DEFAULT 0,
is_active BOOLEAN DEFAULT TRUE,

```
INDEX idx_addons_service (service_id)
);
```

-- Service Bundles

```
CREATE TABLE service_bundles (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(255) NOT NULL,
  description TEXT,
  bundle_price DECIMAL(10, 2) NOT NULL,
  discount_percent DECIMAL(5, 2),
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

  INDEX idx_bundles_active (is_active)
);
```

-- Bundle Services (Many-to-Many)

```
CREATE TABLE bundle_services (
  bundle_id UUID REFERENCES service_bundles(id) ON DELETE CASCADE,
  service_id UUID REFERENCES services(id) ON DELETE CASCADE,
  PRIMARY KEY (bundle_id, service_id)
);
```

-- Partner Services (Services offered by partners with custom pricing)

```
CREATE TABLE partner_services (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  partner_id UUID REFERENCES partners(id) ON DELETE CASCADE,
  service_id UUID REFERENCES services(id) ON DELETE CASCADE,
  custom_price DECIMAL(10, 2) NOT NULL,
  is_available BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

  UNIQUE(partner_id, service_id),
  INDEX idx_partner_services_partner (partner_id),
  INDEX idx_partner_services_service (service_id)
);
```

-- Partner Add-on Pricing

```
CREATE TABLE partner_addon_pricing (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  partner_id UUID REFERENCES partners(id) ON DELETE CASCADE,
  addon_id UUID REFERENCES service_addons(id) ON DELETE CASCADE,
  custom_price DECIMAL(10, 2) NOT NULL,
  is_available BOOLEAN DEFAULT TRUE,
```

```
    UNIQUE(partner_id, addon_id),
    INDEX idx_partner_addons_partner (partner_id)
);
```

3.1.4 Addresses & Locations

```
sql

-- User Addresses
CREATE TABLE user_addresses (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  address_type VARCHAR(50) DEFAULT 'home', -- 'home', 'work', 'other'
  house_number VARCHAR(100),
  street_address TEXT NOT NULL,
  landmark VARCHAR(255),
  pincode VARCHAR(10) NOT NULL,
  city VARCHAR(100) NOT NULL,
  state VARCHAR(100) NOT NULL,
  latitude DECIMAL(10, 8),
  longitude DECIMAL(11, 8),
  is_default BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

  INDEX idx_addresses_user (user_id),
  INDEX idx_addresses_pincode (pincode),
  INDEX idx_addresses_location (latitude, longitude)
);

-- Serviceable Locations
CREATE TABLE serviceable_locations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  pincode VARCHAR(10) UNIQUE NOT NULL,
  city VARCHAR(100) NOT NULL,
  state VARCHAR(100) NOT NULL,
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

  INDEX idx_serviceable_pincode (pincode)
);
```

3.1.5 Bookings

```
sql
```

-- Bookings

```
CREATE TABLE bookings (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  booking_number VARCHAR(50) UNIQUE NOT NULL, -- Human-readable: GRM-2024-001234
```

-- User Info

```
  user_id UUID REFERENCES users(id), -- NULL for guest bookings  
  guest_phone VARCHAR(15),  
  guest_name VARCHAR(255),
```

-- Partner Info

```
  partner_id UUID REFERENCES partners(id),
```

-- Service Details

```
  service_id UUID REFERENCES services(id),  
  service_type VARCHAR(50) NOT NULL, -- 'home', 'salon'
```

-- Location

```
  address_id UUID REFERENCES user_addresses(id),  
  salon_id UUID REFERENCES salon_details(id),
```

-- Scheduling

```
  booking_date DATE NOT NULL,  
  time_slot VARCHAR(50) NOT NULL, -- 'morning', 'afternoon', 'evening'  
  preferred_start_time TIME,  
  actual_start_time TIMESTAMP,  
  actual_end_time TIMESTAMP,
```

-- Pricing

```
  service_base_price DECIMAL(10, 2) NOT NULL,  
  addons_price DECIMAL(10, 2) DEFAULT 0.0,  
  surge_multiplier DECIMAL(4, 2) DEFAULT 1.0,  
  discount_amount DECIMAL(10, 2) DEFAULT 0.0,  
  total_amount DECIMAL(10, 2) NOT NULL,  
  partner_earning DECIMAL(10, 2) NOT NULL,  
  commission_amount DECIMAL(10, 2) NOT NULL,  
  commission_percent DECIMAL(5, 2) NOT NULL,  
  adjustments DECIMAL(10, 2) DEFAULT 0.0, -- penalties, bonuses  
  net_payout_amount DECIMAL(10, 2) NOT NULL,
```

-- Bank Details (snapshot at payout time)

```
  bank_account_number VARCHAR(50) NOT NULL,  
  bank_ifsc_code VARCHAR(20) NOT NULL,  
  bank_account_holder VARCHAR(255) NOT NULL,
```

-- Status

```

payout_status VARCHAR(50) NOT NULL DEFAULT 'pending',
-- 'pending', 'processing', 'completed', 'failed', 'on_hold'

-- Transaction
transaction_reference VARCHAR(255),
payment_gateway_response JSONB,

processed_at TIMESTAMP,
completed_at TIMESTAMP,
failed_at TIMESTAMP,
failure_reason TEXT,

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

INDEX idx_payouts_batch (payout_batch_id),
INDEX idx_payouts_partner (partner_id),
INDEX idx_payouts_status (payout_status),
INDEX idx_payouts_period (period_start_date, period_end_date)
);

-- Payout Booking Items (Which bookings are in this payout)
CREATE TABLE payout_booking_items (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  payout_id UUID REFERENCES partner_payouts(id) ON DELETE CASCADE,
  booking_id UUID REFERENCES bookings(id),

  service_amount DECIMAL(10, 2) NOT NULL,
  commission_amount DECIMAL(10, 2) NOT NULL,
  partner_earning DECIMAL(10, 2) NOT NULL,

  INDEX idx_payout_items_payout (payout_id),
  INDEX idx_payout_items_booking (booking_id)
);

```

3.1.8 Reviews & Ratings

```
sql
```

-- Reviews

```
CREATE TABLE reviews (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  booking_id UUID UNIQUE REFERENCES bookings(id) ON DELETE CASCADE,  
  user_id UUID REFERENCES users(id),  
  partner_id UUID REFERENCES partners(id),  
  
  -- Ratings (1-5)  
  overall_rating DECIMAL(2, 1) NOT NULL CHECK (overall_rating >= 1 AND overall_rating <= 5),  
  professionalism_rating DECIMAL(2, 1),  
  quality_rating DECIMAL(2, 1),  
  punctuality_rating DECIMAL(2, 1),  
  
  -- Review  
  review_text TEXT,  
  review_images TEXT[],  
  
  -- Partner Response  
  partner_response TEXT,  
  partner_responded_at TIMESTAMP,  
  
  -- Status  
  is_verified BOOLEAN DEFAULT FALSE,  
  is_visible BOOLEAN DEFAULT TRUE,  
  is_flagged BOOLEAN DEFAULT FALSE,  
  flagged_reason TEXT,  
  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
  INDEX idx_reviews_booking (booking_id),  
  INDEX idx_reviews_user (user_id),  
  INDEX idx_reviews_partner (partner_id),  
  INDEX idx_reviews_rating (overall_rating DESC),  
  INDEX idx_reviews_visible (is_visible, created_at DESC)  
);
```

3.1.9 Referrals & Memberships

sql

-- Referrals

```
CREATE TABLE referrals (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  referrer_user_id UUID REFERENCES users(id),  
  referee_user_id UUID REFERENCES users(id),  
  
  referral_code VARCHAR(50) NOT NULL,  
  
  -- Status  
  referral_status VARCHAR(50) NOT NULL DEFAULT 'pending',  
  -- 'pending', 'completed', 'expired', 'cancelled'  
  
  -- Rewards  
  referrer_reward_amount DECIMAL(10, 2) DEFAULT 100.00,  
  referee_reward_amount DECIMAL(10, 2) DEFAULT 100.00,  
  
  referrer_credited BOOLEAN DEFAULT FALSE,  
  referee_credited BOOLEAN DEFAULT FALSE,  
  
  referrer_credited_at TIMESTAMP,  
  referee_credited_at TIMESTAMP,  
  
  -- First booking by referee  
  first_booking_id UUID REFERENCES bookings(id),  
  completed_at TIMESTAMP,  
  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  expires_at TIMESTAMP,  
  
  INDEX idx_referrals_referrer (referrer_user_id),  
  INDEX idx_referrals_referee (referee_user_id),  
  INDEX idx_referrals_code (referral_code),  
  INDEX idx_referrals_status (referral_status)  
);
```

-- User Referral Codes

```
CREATE TABLE user_referral_codes (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID UNIQUE REFERENCES users(id) ON DELETE CASCADE,  
  referral_code VARCHAR(50) UNIQUE NOT NULL,  
  total_referrals INT DEFAULT 0,  
  successful_referrals INT DEFAULT 0,  
  total_earned DECIMAL(10, 2) DEFAULT 0.0,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
  INDEX idx_referral_codes_user (user_id),
```

```

    INDEX idx_referral_codes_code (referral_code)
);

-- Membership Plans
CREATE TABLE membership_plans (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(255) NOT NULL,
    description TEXT,

    -- Pricing
    monthly_price DECIMAL(10, 2) NOT NULL,
    validity_days INT NOT NULL,

    -- Benefits
    discount_percent DECIMAL(5, 2),
    flat_discount_amount DECIMAL(10, 2),
    priority_support BOOLEAN DEFAULT FALSE,
    priority_booking BOOLEAN DEFAULT FALSE,

    -- Limits
    max_bookings_per_month INT,

    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    INDEX idx_membership_plans_active (is_active)
);

-- User Memberships
CREATE TABLE user_memberships (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(id),
    plan_id UUID REFERENCES membership_plans(id),

    -- Subscription
    subscription_status VARCHAR(50) NOT NULL DEFAULT 'active',
    -- 'active', 'expired', 'cancelled', 'paused'

    started_at TIMESTAMP NOT NULL,
    expires_at TIMESTAMP NOT NULL,
    cancelled_at TIMESTAMP,

    -- Payment
    payment_id UUID REFERENCES payments(id),

    -- Usage

```

```
bookings_used INT DEFAULT 0,  
  
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
INDEX idx_memberships_user (user_id),  
INDEX idx_memberships_status (subscription_status),  
INDEX idx_memberships_expires (expires_at)  
);
```

3.1.10 Coupons & Promotions

sql

-- Coupons

```
CREATE TABLE coupons (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  code VARCHAR(50) UNIQUE NOT NULL,  
  
  -- Details  
  title VARCHAR(255) NOT NULL,  
  description TEXT,  
  
  -- Discount  
  discount_type VARCHAR(50) NOT NULL, -- 'percentage', 'fixed'  
  discount_value DECIMAL(10, 2) NOT NULL,  
  max_discount_amount DECIMAL(10, 2),  
  min_order_amount DECIMAL(10, 2),  
  
  -- Usage Limits  
  usage_limit_per_user INT DEFAULT 1,  
  total_usage_limit INT,  
  current_usage_count INT DEFAULT 0,  
  
  -- Validity  
  valid_from TIMESTAMP NOT NULL,  
  valid_until TIMESTAMP NOT NULL,  
  
  -- Restrictions  
  applicable_service_ids UUID[],  
  applicable_user_ids UUID[],  
  first_order_only BOOLEAN DEFAULT FALSE,  
  
  is_active BOOLEAN DEFAULT TRUE,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
  INDEX idx_coupons_code (code),  
  INDEX idx_coupons_active (is_active),  
  INDEX idx_coupons_validity (valid_from, valid_until)  
);
```

-- Coupon Usage

```
CREATE TABLE coupon_usage (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  coupon_id UUID REFERENCES coupons(id),  
  user_id UUID REFERENCES users(id),  
  booking_id UUID REFERENCES bookings(id),  
  
  discount_amount DECIMAL(10, 2) NOT NULL,
```

```
used_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
INDEX idx_coupon_usage_coupon (coupon_id),
```

```
INDEX idx_coupon_usage_user (user_id),
```

```
INDEX idx_coupon_usage_booking (booking_id)
```

```
);
```

3.1.11 Notifications

```
sql
```

-- Notifications

```
CREATE TABLE notifications (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
  -- Recipient  
  user_id UUID REFERENCES users(id),  
  partner_id UUID REFERENCES partners(id),  
  
  -- Notification Details  
  notification_type VARCHAR(50) NOT NULL,  
  -- 'booking_confirmed', 'partner_assigned', 'service_started', etc.  
  
  title VARCHAR(255) NOT NULL,  
  message TEXT NOT NULL,  
  
  -- Channels  
  channels TEXT[] NOT NULL, -- ['sms', 'email', 'whatsapp', 'push']  
  
  -- Delivery Status per Channel  
  delivery_status JSONB,  
  -- {"sms": {"status": "sent", "sent_at": "..."}, "email": {...}}  
  
  -- Metadata  
  booking_id UUID REFERENCES bookings(id),  
  metadata JSONB,  
  
  -- Status  
  is_read BOOLEAN DEFAULT FALSE,  
  read_at TIMESTAMP,  
  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
  INDEX idx_notifications_user (user_id),  
  INDEX idx_notifications_partner (partner_id),  
  INDEX idx_notifications_type (notification_type),  
  INDEX idx_notifications_read (is_read, created_at DESC)  
);
```

-- Notification Templates

```
CREATE TABLE notification_templates (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  template_key VARCHAR(100) UNIQUE NOT NULL,  
  template_name VARCHAR(255) NOT NULL,  
  
  -- Channel-specific templates  
  sms_template TEXT,
```

```

email_subject TEXT,
email_template TEXT,
whatsapp_template TEXT,
push_title TEXT,
push_body TEXT,

-- Variables in template
template_variables TEXT[],

is_active BOOLEAN DEFAULT TRUE,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

INDEX idx_templates_key (template_key)
);

-- Push Notification Tokens
CREATE TABLE push_notification_tokens (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id),
  partner_id UUID REFERENCES partners(id),

  token VARCHAR(500) NOT NULL,
  device_type VARCHAR(50), -- 'android', 'ios', 'web'
  device_info JSONB,

  is_active BOOLEAN DEFAULT TRUE,
  last_used_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

  INDEX idx_push_tokens_user (user_id),
  INDEX idx_push_tokens_partner (partner_id),
  INDEX idx_push_tokens_active (is_active)
);

```

3.1.12 Admin & System

```
sql
```

-- Admin Users

```
CREATE TABLE admin_users (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  email VARCHAR(255) UNIQUE NOT NULL,  
  password_hash VARCHAR(255) NOT NULL,  
  full_name VARCHAR(255) NOT NULL,  
  
  -- Role & Permissions  
  role VARCHAR(50) NOT NULL DEFAULT 'admin',  
  -- 'super_admin', 'admin', 'finance', 'support', 'operations'  
  
  permissions JSONB, -- Granular permissions  
  
  is_active BOOLEAN DEFAULT TRUE,  
  last_login_at TIMESTAMP,  
  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
  INDEX idx_admin_email (email)  
);
```

-- Audit Logs

```
CREATE TABLE audit_logs (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
  -- Actor  
  admin_user_id UUID REFERENCES admin_users(id),  
  user_id UUID REFERENCES users(id),  
  partner_id UUID REFERENCES partners(id),  
  
  -- Action  
  action VARCHAR(100) NOT NULL,  
  entity_type VARCHAR(100) NOT NULL,  
  entity_id UUID,  
  
  -- Details  
  old_values JSONB,  
  new_values JSONB,  
  metadata JSONB,  
  
  ip_address INET,  
  user_agent TEXT,  
  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```



```

INDEX idx_audit_entity (entity_type, entity_id),
INDEX idx_audit_admin (admin_user_id),
INDEX idx_audit_action (action),
INDEX idx_audit_created (created_at DESC)
);

-- System Settings
CREATE TABLE system_settings (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  setting_key VARCHAR(100) UNIQUE NOT NULL,
  setting_value JSONB NOT NULL,
  description TEXT,

  updated_by UUID REFERENCES admin_users(id),
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

  INDEX idx_settings_key (setting_key)
);

-- Example settings:
-- 'commission_percent': 20
-- 'job_request_timeout_seconds': 20
-- 'surge_pricing_config': {...}
-- 'payment_config': {...}

```

3.2 Database Indexes Strategy

sql

-- Additional Performance Indexes

-- Composite indexes for common queries

```
CREATE INDEX idx_bookings_partner_status_date
ON bookings(partner_id, status, booking_date);
```

```
CREATE INDEX idx_bookings_user_status_date
ON bookings(user_id, status, booking_date DESC);
```

-- Full-text search indexes

```
CREATE INDEX idx_services_search
ON services USING gin(to_tsvector('english', name || ' ' || description));
```

```
CREATE INDEX idx_partners_search
ON partners USING gin(to_tsvector('english', full_name || ' ' || COALESCE(bio, '')));
```

-- GIN indexes for JSONB columns

```
CREATE INDEX idx_notifications_delivery_status
ON notifications USING gin(delivery_status);
```

-- Partial indexes for active records only

```
CREATE INDEX idx_partners_active_location
ON partners(latitude, longitude)
WHERE is_active = true AND verification_status = 'verified';
```

```
CREATE INDEX idx_services_active
ON services(subcategory_id, display_order)
WHERE is_active = true;
```

3.3 Database Constraints & Triggers

sql

-- Constraints

ALTER TABLE bookings

ADD CONSTRAINT booking_user_or_guest

CHECK ((user_id IS NOT NULL) OR (guest_phone IS NOT NULL AND guest_name IS NOT NULL));

ALTER TABLE bookings

ADD CONSTRAINT booking_location

CHECK ((service_type = 'home' AND address_id IS NOT NULL) OR
(service_type = 'salon' AND salon_id IS NOT NULL));

-- Triggers for updated_at

CREATE OR REPLACE FUNCTION update_updated_at_column()

RETURNS TRIGGER AS \$

BEGIN

NEW.updated_at = CURRENT_TIMESTAMP;

RETURN NEW;

END;

\$ LANGUAGE plpgsql;

CREATE TRIGGER update_users_updated_at BEFORE UPDATE ON users

FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();

-- Apply to all tables with updated_at column

-- (repeat for partners, bookings, services, etc.)

-- Trigger to update partner ratings

CREATE OR REPLACE FUNCTION update_partner_rating()

RETURNS TRIGGER AS \$

BEGIN

UPDATE partners

SET

average_rating = (

SELECT AVG(overall_rating)

FROM reviews

WHERE partner_id = NEW.partner_id AND is_visible = true

),

total_ratings = (

SELECT COUNT(*)

FROM reviews

WHERE partner_id = NEW.partner_id AND is_visible = true

)

WHERE id = NEW.partner_id;

RETURN NEW;

END;

\$ LANGUAGE plpgsql;

```

CREATE TRIGGER update_partner_rating_trigger
AFTER INSERT OR UPDATE ON reviews
FOR EACH ROW EXECUTE FUNCTION update_partner_rating();

-- Trigger to validate wallet balance
CREATE OR REPLACE FUNCTION validate_wallet_transaction()
RETURNS TRIGGER AS $
BEGIN
    IF NEW.transaction_type = 'debit' THEN
        IF NEW.balance_before < NEW.amount THEN
            RAISE EXCEPTION 'Insufficient wallet balance';
        END IF;
    END IF;
    RETURN NEW;
END;
$ LANGUAGE plpgsql;

CREATE TRIGGER validate_wallet_debit
BEFORE INSERT ON wallet_transactions
FOR EACH ROW EXECUTE FUNCTION validate_wallet_transaction();

```

4. API Structure

4.1 API Versioning

- Base URL: `https://api.groomsta.com/v1`
- Versioning strategy: URI versioning for major changes

4.2 RESTful API Endpoints

4.2.1 Authentication & Authorization

typescript

// Auth Module Endpoints

POST /auth/send-otp // Send OTP for login/registration
POST /auth/verify-otp // Verify OTP and issue tokens
POST /auth/login // Email/password login
POST /auth/refresh-token // Refresh access token
POST /auth/logout // Logout (invalidate session)

// Partner Auth

POST /auth/partner/send-otp
POST /auth/partner/verify-otp
POST /auth/partner/login
POST /auth/partner/refresh-token
POST /auth/partner/logout

// Admin Auth

POST /auth/admin/login
POST /auth/admin/logout
POST /auth/admin/refresh-token

4.2.2 User Management

typescript

// Users Module

GET /users/me // Get current user profile
PUT /users/me // Update user profile
DELETE /users/me // Delete account

// User Addresses

GET /users/me/addresses // List user addresses
POST /users/me/addresses // Add new address
PUT /users/me/addresses/:id // Update address
DELETE /users/me/addresses/:id // Delete address
PUT /users/me/addresses/:id/default // Set default address

// User Wallet

GET /users/me/wallet // Get wallet balance
GET /users/me/wallet/transactions // List wallet transactions

// User Bookings

GET /users/me/bookings // List user bookings
GET /users/me/bookings/:id // Get booking details

// User Reviews

GET /users/me/reviews // List user reviews
POST /bookings/:id/review // Submit review for booking

// User Referrals

GET /users/me/referral-code // Get referral code
GET /users/me/referrals // List referrals and earnings

4.2.3 Partner Management

typescript

// Partner Onboarding

POST /partners/register // Initial registration
POST /partners/documents/upload // Upload KYC documents
PUT /partners/profile // Update profile
PUT /partners/bank-details // Update bank details
PUT /partners/salon-details // Update salon details (if applicable)

// Partner Profile

GET /partners/me // Get partner profile
PUT /partners/me // Update partner profile
GET /partners/me/verification-status // Check verification status

// Partner Services

GET /partners/me/services // List partner services
PUT /partners/me/services/:id/pricing // Update service pricing
PUT /partners/me/services/:id/toggle // Enable/disable service

// Partner Availability

GET /partners/me/availability // Get availability schedule
PUT /partners/me/availability // Update availability
POST /partners/me/blocked-dates // Block specific dates
DELETE /partners/me/blocked-dates/:id // Unblock date

// Partner Jobs

GET /partners/me/jobs/incoming // List incoming job requests
GET /partners/me/jobs/active // List active jobs
GET /partners/me/jobs/past // List past jobs
GET /partners/me/jobs/:id // Get job details
POST /partners/me/jobs/:id/accept // Accept job request
POST /partners/me/jobs/:id/reject // Reject job request

// Job Status Updates

PUT /bookings/:id/status/on-the-way // Update status: on the way
PUT /bookings/:id/status/started // Update status: service started
PUT /bookings/:id/status/completed // Update status: completed

// Partner Earnings

GET /partners/me/earnings // Current week earnings
GET /partners/me/earnings/summary // Earnings summary
GET /partners/me/payouts // Payout history
GET /partners/me/payouts/:id // Payout details

// Partner Reviews

GET /partners/me/reviews // List reviews received
PUT /partners/me/reviews/:id/response // Respond to review

4.2.4 Services Catalog

typescript

// Public Service Browsing

GET /services/categories // List all categories
GET /services/categories/:slug // Get category with subcategories
GET /services/subcategories/:slug // Get subcategory with services
GET /services/:slug // Get service details
GET /services/search // Search services

// Service Details

GET /services/:id/addons // Get service add-ons
GET /services/:id/bundles // Get related bundles
GET /services/:id/reviews // Get service reviews

4.2.5 Booking Management

typescript

// Booking Creation

POST /bookings/check-availability // Check service availability
POST /bookings/calculate-price // Calculate booking price
POST /bookings // Create new booking

// Booking Management

GET /bookings/:id // Get booking details
PUT /bookings/:id/cancel // Cancel booking
GET /bookings/:id/status // Get booking status
GET /bookings/:id/partner-location // Get partner real-time location (future)

// Guest Booking

POST /bookings/guest // Create guest booking
GET /bookings/guest/:bookingNumber // Get guest booking by number

4.2.6 Payments

typescript

// Payment Initiation

POST /payments/create-order // Create Razorpay order

POST /payments/verify // Verify payment

POST /payments/wallet/apply // Apply wallet balance

// Payment Status

GET /payments/:id // Get payment details

GET /bookings/:id/payments // List booking payments

// Refunds

POST /payments/:id/refund // Request refund

GET /refunds/:id // Get refund status

4.2.7 Coupons & Promotions

typescript

// Coupons

POST /coupons/validate // Validate coupon code

POST /coupons/apply // Apply coupon to booking

GET /coupons/available // List available coupons

4.2.8 Memberships

typescript

// Membership Plans

GET /memberships/plans // List membership plans

GET /memberships/plans/:id // Get plan details

// User Membership

POST /memberships/subscribe // Subscribe to plan

GET /users/me/membership // Get current membership

PUT /users/me/membership/cancel // Cancel membership

4.2.9 Referrals

typescript

// Referral System

GET /referrals/my-code // Get user's referral code

POST /referrals/apply // Apply referral code

GET /referrals/stats // Get referral statistics

4.2.10 Admin APIs

// Dashboard

GET /admin/dashboard/stats // Dashboard statistics

GET /admin/dashboard/metrics // Key metrics

// Service Management

GET /admin/services // List all services

POST /admin/services // Create service

PUT /admin/services/:id // Update service

DELETE /admin/services/:id // Delete service

GET /admin/categories // List categories

POST /admin/categories // Create category

PUT /admin/categories/:id // Update category

// Partner Management

GET /admin/partners // List all partners

GET /admin/partners/pending // List pending verifications

GET /admin/partners/:id // Get partner details

PUT /admin/partners/:id/verify // Verify partner

PUT /admin/partners/:id/reject // Reject partner

PUT /admin/partners/:id/suspend // Suspend partner

PUT /admin/partners/:id/activate // Activate partner

// Booking Management

GET /admin/bookings // List all bookings

GET /admin/bookings/:id // Get booking details

PUT /admin/bookings/:id // Update booking

POST /admin/bookings/:id/refund // Process refund

// Payout Management

GET /admin/payouts/pending // Pending payouts

POST /admin/payouts/generate // Generate payout batch

PUT /admin/payouts/process/:batchId // Process payout batch

GET /admin/payouts/history // Payout history

// Coupon Management

GET /admin/coupons // List coupons

POST /admin/coupons // Create coupon

PUT /admin/coupons/:id // Update coupon

DELETE /admin/coupons/:id // Delete coupon

// User Management

GET /admin/users // List users

GET /admin/users/:id // Get user details

PUT /admin/users/:id/block // Block user

// Analytics & Reports

GET	/admin/reports/revenue	// Revenue reports
GET	/admin/reports/bookings	// Booking reports
GET	/admin/reports/partners	// Partner performance
GET	/admin/reports/customers	// Customer metrics
POST	/admin/reports/export	// Export data

// System Settings

GET	/admin/settings	// List settings
PUT	/admin/settings/:key	// Update setting

4.3 API Response Format

typescript

// Success Response

```
{
  "success": true,
  "data": { /* response data */ },
  "message": "Operation successful",
  "metadata": {
    "timestamp": "2024-12-03T10:30:00Z",
    "requestId": "uuid"
  }
}
```

// Paginated Response

```
{
  "success": true,
  "data": [ /* items */ ],
  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 150,
    "totalPages": 8,
    "hasNext": true,
    "hasPrevious": false
  },
  "metadata": { /* ... */ }
}
```

// Error Response

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid input data",
    "details": [
      {
        "field": "phone",
        "message": "Phone number is required"
      }
    ]
  },
  "metadata": { /* ... */ }
}
```

4.4 API Request/Response Examples

typescript

// Example: Create Booking

POST /bookings

Authorization: Bearer {token}

Request Body:

```
{
  "serviceId": "uuid",
  "serviceType": "home",
  "addressId": "uuid",
  "bookingDate": "2024-12-10",
  "timeSlot": "morning",
  "addons": ["uuid1", "uuid2"],
  "paymentType": "full",
  "customerNotes": "Please bring professional equipment"
}
```

Response (201):

```
{
  "success": true,
  "data": {
    "bookingId": "uuid",
    "bookingNumber": "GRM-2024-001234",
    "status": "pending",
    "estimatedPrice": {
      "min": 500,
      "max": 700
    },
    "paymentOrderId": "order_xyz123",
    "expiresAt": "2024-12-03T10:40:00Z"
  },
  "message": "Booking created successfully"
}
```

5. Authentication & Authorization

5.1 JWT Token Strategy

typescript

// Token Structure

```
interface AccessToken {  
  sub: string;           // User/Partner ID  
  type: 'user' | 'partner' | 'admin';  
  email?: string;  
  phone?: string;  
  role?: string;         // For admin  
  iat: number;           // Issued at  
  exp: number;           // Expires at (15 minutes)  
}
```

```
interface RefreshToken {  
  sub: string;  
  type: 'user' | 'partner' | 'admin';  
  sessionId: string;  
  iat: number;  
  exp: number;           // Expires at (7 days)  
}
```

// Token Generation

```
const generateTokens = async (userId: string, type: string) => {  
  const accessToken = jwt.sign(  
    { sub: userId, type },  
    process.env.JWT_SECRET,  
    { expiresIn: '15m' }  
  );  
  
  const refreshToken = jwt.sign(  
    { sub: userId, type, sessionId: generateUUID() },  
    process.env.JWT_REFRESH_SECRET,  
    { expiresIn: '7d' }  
  );  
  
  // Store refresh token in database  
  await storeRefreshToken(userId, refreshToken);  
  
  return { accessToken, refreshToken };  
};
```

5.2 Authorization Guards

typescript

```

// Role-based Access Control
enum UserRole {
    CUSTOMER = 'customer',
    PARTNER = 'partner',
    ADMIN = 'admin',
    SUPER_ADMIN = 'super_admin'
}

// Guard Decorator
@UseGuards(JwtAuthGuard, RolesGuard)
@Roles(UserRole.ADMIN, UserRole.SUPER_ADMIN)
@Get('/admin/dashboard')
async getDashboard() {
    // Only accessible by admin roles
}

// Permission-based Access Control
enum Permission {
    // Bookings
    VIEW_BOOKINGS = 'view_bookings',
    MANAGE_BOOKINGS = 'manage_bookings',
    CANCEL_BOOKINGS = 'cancel_bookings',

    // Partners
    VIEW_PARTNERS = 'view_partners',
    VERIFY_PARTNERS = 'verify_partners',
    MANAGE_PARTNERS = 'manage_partners',

    // Payouts
    VIEW_PAYOUTS = 'view_payouts',
    PROCESS_PAYOUTS = 'process_payouts',

    // Settings
    MANAGE_SETTINGS = 'manage_settings'
}

@UseGuards(PermissionGuard)
@RequirePermissions(Permission.PROCESS_PAYOUTS)
@Post('/admin/payouts/process/:batchId')
async processPayouts(@Param('batchId') batchId: string) {
    // Only accessible by users with payout processing permission
}

```

5.3 OTP System

// OTP Service

```
class OtpService {  
  async sendOtp(phone: string, purpose: string): Promise<void> {  
    // Generate 6-digit OTP  
    const otp = Math.floor(100000 + Math.random() * 900000).toString();  
  
    // Store in database with expiration (5 minutes)  
    await this.otpRepository.save({  
      phone,  
      otp: await bcrypt.hash(otp, 10), // Hash OTP  
      purpose,  
      expiresAt: new Date(Date.now() + 5 * 60 * 1000),  
      attempts: 0  
    });  
  
    // Send via SMS gateway  
    await this.smsService.send(phone, `Your Groomsta OTP is: ${otp}. Valid for 5 minutes.`);  
  
    // Rate limiting: max 3 OTPs per phone per hour  
    await this.redis.incr(`otp:ratelimit:${phone}`);  
    await this.redis.expire(`otp:ratelimit:${phone}`, 3600);  
  }  
  
  async verifyOtp(phone: string, otp: string, purpose: string): Promise<boolean> {  
    const record = await this.otpRepository.findOne({  
      where: { phone, purpose, isUsed: false },  
      order: { createdAt: 'DESC' }  
    });  
  
    if (!record || new Date() > record.expiresAt) {  
      throw new BadRequestException('OTP expired or invalid');  
    }  
  
    // Max 3 attempts  
    if (record.attempts >= 3) {  
      throw new BadRequestException('Maximum OTP attempts exceeded');  
    }  
  
    // Verify OTP  
    const isValid = await bcrypt.compare(otp, record.otp);  
  
    if (!isValid) {  
      await this.otpRepository.update(record.id, {  
        attempts: record.attempts + 1  
      });  
      throw new BadRequestException('Invalid OTP');  
    }  
  }  
}
```

```
}

// Mark as used
await this.otpRepository.update(record.id, { isUsed: true });

return true;
}
```

6. Real-time Communication

6.1 WebSocket Architecture

typescript

```

// Socket.io Gateway
@WebSocketGateway({
  cors: { origin: '*' },
  namespace: '/ws'
})

export class EventsGateway implements OnGatewayConnection, OnGatewayDisconnect {
  @WebSocketServer()
  server: Server;

  constructor(
    private readonly jwtService: JwtService,
    private readonly redis: Redis
  ) {}

  async handleConnection(client: Socket) {
    try {
      // Authenticate socket connection
      const token = client.handshake.auth.token;
      const payload = await this.jwtService.verify(token);

      // Join user/partner-specific room
      if (payload.type === 'partner') {
        client.join(`partner:${payload.sub}`);
        await this.redis.set(`partner:online:${payload.sub}`, '1', 'EX', 3600);
      } else if (payload.type === 'user') {
        client.join(`user:${payload.sub}`);
      }

      client.data.userId = payload.sub;
      client.data.userType = payload.type;

    } catch (error) {
      client.disconnect();
    }
  }

  async handleDisconnect(client: Socket) {
    if (client.data.userType === 'partner') {
      await this.redis.del(`partner:online:${client.data.userId}`);
    }
  }

  // Emit job request to nearby partners
  async emitJobRequest(partnerIds: string[], bookingData: any) {
    partnerIds.forEach(partnerId => {
      this.server.to(`partner:${partnerId}`).emit('job:request', {

```

```

        bookingId: bookingData.id,
        service: bookingData.service,
        location: bookingData.location,
        estimatedEarning: bookingData.partnerEarning,
        expiresAt: bookingData.expiresAt
    });
});
}

// Emit booking status updates to customer
emitBookingStatusUpdate(userId: string, bookingId: string, status: string, data: any) {
    this.server.to(`user:${userId}`).emit('booking:status', {
        bookingId,
        status,
        ...data
    });
}

// Emit partner location updates
emitPartnerLocation(userId: string, location: { latitude: number; longitude: number }) {
    this.server.to(`user:${userId}`).emit('partner:location', location);
}
}

```

6.2 Real-time Events

typescript

```
// Event Types
```

```
enum SocketEvent {
```

```
  // Job Events (Partner)
```

```
  JOB_REQUEST = 'job:request',
```

```
  JOB_ACCEPTED = 'job:accepted',
```

```
  JOB_CANCELLED = 'job:cancelled',
```

```
  // Booking Events (Customer)
```

```
  BOOKING_STATUS = 'booking:status',
```

```
  PARTNER_ASSIGNED = 'booking:partner_assigned',
```

```
  PARTNER_LOCATION = 'partner:location',
```

```
  // General
```

```
  NOTIFICATION = 'notification',
```

```
  PING = 'ping',
```

```
  PONG = 'pong'
```

```
}
```

```
// Client Event Handlers
```

```
@SubscribeMessage('partner:accept_job')
```

```
async handleAcceptJob(
```

```
  @MessageBody() data: { bookingId: string },
```

```
  @ConnectedSocket() client: Socket
```

```
) {
```

```
  const partnerId = client.data.userId;
```

```
  // Process job acceptance
```

```
  const result = await this.bookingService.acceptJob(data.bookingId, partnerId);
```

```
  if (result.success) {
```

```
    // Notify customer
```

```
    this.emitBookingStatusUpdate(
```

```
      result.booking.userId,
```

```
      data.bookingId,
```

```
      'partner_assigned',
```

```
      { partner: result.partner }
```

```
    );
```

```
    // Notify other partners that job is taken
```

```
    result.rejectedPartners.forEach(pId => {
```

```
      this.server.to(`partner:${pId}`).emit('job:taken', { bookingId: data.bookingId });
```

```
    });
```

```
  }
```

```
  return result;
```

```
}
```

```

@SubscribeMessage('partner:update_location')
async handleLocationUpdate(
  @MessageBody() data: { latitude: number; longitude: number; bookingId: string },
  @ConnectedSocket() client: Socket
) {
  const partnerId = client.data.userId;

  // Update partner location in Redis
  await this.redis.set(
    `partner:location:${partnerId}`,
    JSON.stringify({ lat: data.latitude, lng: data.longitude }),
    'EX',
    300
  );

  // Emit to customer if they're tracking this booking
  const booking = await this.bookingService.findOne(data.bookingId);
  if (booking && booking.userId) {
    this.emitPartnerLocation(booking.userId, {
      latitude: data.latitude,
      longitude: data.longitude
    });
  }
}

```

6.3 Redis Pub/Sub for Multi-Server

typescript

// For horizontal scaling across multiple servers

```
class RedisEventBus {
  private publisher: Redis;
  private subscriber: Redis;

  constructor() {
    this.publisher = new Redis(config.redis);
    this.subscriber = new Redis(config.redis);

    this.setupSubscriptions();
  }

  async publish(channel: string, data: any) {
    await this.publisher.publish(channel, JSON.stringify(data));
  }

  private setupSubscriptions() {
    this.subscriber.subscribe('job:requests', 'booking:updates', 'notifications');

    this.subscriber.on('message', (channel, message) => {
      const data = JSON.parse(message);

      switch(channel) {
        case 'job:requests':
          // Emit to connected partners on this server instance
          this.handleJobRequest(data);
          break;
        case 'booking:updates':
          this.handleBookingUpdate(data);
          break;
        case 'notifications':
          this.handleNotification(data);
          break;
      }
    });
  }
}
```

7. Payment Integration

7.1 Razorpay Integration

typescript

// Razorpay Service

```
class RazorpayService {  
  private razorpay: Razorpay;  
  
  constructor() {  
    this.razorpay = new Razorpay({  
      key_id: process.env.RAZORPAY_KEY_ID,  
      key_secret: process.env.RAZORPAY_KEY_SECRET  
    });  
  }  
}
```

// Create order

```
async createOrder(amount: number, bookingId: string, metadata: any) {  
  const order = await this.razorpay.orders.create({  
    amount: amount * 100, // Convert to paise  
    currency: 'INR',  
    receipt: `booking_${bookingId}`,  
    notes: {  
      bookingId,  
      ...metadata  
    }  
  });  
}
```

// Store order in database

```
await this.paymentRepository.save({  
  bookingId,  
  paymentGateway: 'razorpay',  
  gatewayOrderId: order.id,  
  amount,  
  paymentStatus: 'pending'  
});
```

```
return {  
  orderId: order.id,  
  amount: order.amount,  
  currency: order.currency  
};  
}
```

// Verify payment

```
async verifyPayment(  
  orderId: string,  
  paymentId: string,  
  signature: string  
): Promise<boolean> {  
  const text = `${orderId}|${paymentId}`;
```



```

const expectedSignature = crypto
  .createHmac('sha256', process.env.RAZORPAY_KEY_SECRET)
  .update(text)
  .digest('hex');

if (signature !== expectedSignature) {
  throw new BadRequestException('Invalid payment signature');
}

// Update payment record
await this.paymentRepository.update(
  { gatewayOrderId: orderId },
  {
    gatewayPaymentId: paymentId,
    paymentStatus: 'success',
    completedAt: new Date()
  }
);

return true;
}

// Process refund
async processRefund(paymentId: string, amount: number, notes: any) {
  const payment = await this.paymentRepository.findOne({
    where: { gatewayPaymentId: paymentId }
  });

  if (!payment) {
    throw new NotFoundException('Payment not found');
  }

  const refund = await this.razorpay.payments.refund(paymentId, {
    amount: amount * 100,
    notes
  });

  // Create refund record
  await this.refundRepository.save({
    paymentId: payment.id,
    bookingId: payment.bookingId,
    userId: payment.userId,
    refundAmount: amount,
    refundType: amount === payment.amount ? 'full' : 'partial',
    refundMethod: 'original_source',
    gatewayRefundId: refund.id,
    gatewayResponse: refund,
  });
}

```

```

    refundStatus: 'processing',
    initiatedBy: 'system'
  });

  return refund;
}

// Webhook handler
async handleWebhook(payload: any, signature: string) {
  // Verify webhook signature
  const isValid = this.verifyWebhookSignature(payload, signature);
  if (!isValid) {
    throw new BadRequestException('Invalid webhook signature');
  }

  const event = payload.event;

  switch(event) {
    case 'payment.captured':
      await this.handlePaymentCaptured(payload.payload.payment.entity);
      break;
    case 'payment.failed':
      await this.handlePaymentFailed(payload.payload.payment.entity);
      break;
    case 'refund.created':
      await this.handleRefundCreated(payload.payload.refund.entity);
      break;
    case 'refund.processed':
      await this.handleRefundProcessed(payload.payload.refund.entity);
      break;
  }
}

private verifyWebhookSignature(payload: any, signature: string): boolean {
  const expectedSignature = crypto
    .createHmac('sha256', process.env.RAZORPAY_WEBHOOK_SECRET)
    .update(JSON.stringify(payload))
    .digest('hex');

  return signature === expectedSignature;
}
}

```

7.2 Wallet Management

typescript

```
// Wallet Service
```

```
class WalletService {  
  async credit(  
    userId: string,  
    amount: number,  
    transactionType: string,  
    metadata: any  
  ) {  
    return await this.db.transaction(async (entityManager) => {  
      // Get or create wallet  
      let wallet = await entityManager.findOne(UserWallet, {  
        where: { userId },  
        lock: { mode: 'pessimistic_write' } // Lock for concurrent safety  
      });  
  
      if (!wallet) {  
        wallet = await entityManager.save(UserWallet, {  
          userId,  
          balance: 0,  
          totalCredited: 0,  
          totalDebited: 0  
        });  
      }  
  
      const balanceBefore = wallet.balance;  
      const balanceAfter = balanceBefore + amount;  
  
      // Update wallet  
      await entityManager.update(UserWallet, wallet.id, {  
        balance: balanceAfter,  
        totalCredited: wallet.totalCredited + amount  
      });  
  
      // Create transaction record  
      const transaction = await entityManager.save(WalletTransaction, {  
        walletId: wallet.id,  
        userId,  
        transactionType,  
        amount,  
        balanceBefore,  
        balanceAfter,  
        ...metadata  
      });  
  
      return transaction;  
    });  
  }  
}
```

```
}
```

```
async debit(
  userId: string,
  amount: number,
  transactionType: string,
  metadata: any
) {
  return await this.db.transaction(async (entityManager) => {
    const wallet = await entityManager.findOne(UserWallet, {
      where: { userId },
      lock: { mode: 'pessimistic_write' }
    });

    if (!wallet || wallet.balance < amount) {
      throw new BadRequestException('Insufficient wallet balance');
    }

    const balanceBefore = wallet.balance;
    const balanceAfter = balanceBefore - amount;

    // Update wallet
    await entityManager.update(UserWallet, wallet.id, {
      balance: balanceAfter,
      totalDebited: wallet.totalDebited + amount
    });

    // Create transaction record
    const transaction = await entityManager.save(WalletTransaction, {
      walletId: wallet.id,
      userId,
      transactionType,
      amount,
      balanceBefore,
      balanceAfter,
      ...metadata
    });

    return transaction;
  });
}

async getBalance(userId: string): Promise<number> {
  const wallet = await this.walletRepository.findOne({ where: { userId } });
  return wallet ? wallet.balance : 0;
}
```

8. Notification System

8.1 Multi-Channel Notification Service

typescript

// Notification Service

```
class NotificationService {  
  constructor(  
    private readonly smsService: SmsService,  
    private readonly emailService: EmailService,  
    private readonly whatsappService: WhatsappService,  
    private readonly pushService: PushNotificationService,  
    private readonly templateService: TemplateService  
  ) {}  
  
  async send(notification: NotificationDto) {  
    const { recipientId, recipientType, type, channels, data } = notification;  
  
    // Get template  
    const template = await this.templateService.getTemplate(type);  
  
    // Get recipient contact info  
    const recipient = await this.getRecipient(recipientId, recipientType);  
  
    // Prepare delivery status  
    const deliveryStatus: any = {};  
  
    // Send via each channel  
    const promises = channels.map(async (channel) => {  
      try {  
        switch(channel) {  
          case 'sms':  
            if (recipient.phone) {  
              const message = this.templateService.render(template.smsTemplate, data);  
              await this.smsService.send(recipient.phone, message);  
              deliveryStatus.sms = { status: 'sent', sentAt: new Date() };  
            }  
            break;  
  
          case 'email':  
            if (recipient.email) {  
              const subject = this.templateService.render(template.emailSubject, data);  
              const body = this.templateService.render(template.emailTemplate, data);  
              await this.emailService.send(recipient.email, subject, body);  
              deliveryStatus.email = { status: 'sent', sentAt: new Date() };  
            }  
            break;  
  
          case 'whatsapp':  
            if (recipient.phone) {  
              const message = this.templateService.render(template.whatsappTemplate, data);
```

```

        await this.whatsappService.send(recipient.phone, message);
        deliveryStatus.whatsapp = { status: 'sent', sentAt: new Date() };
    }
    break;

    case 'push':
        const tokens = await this.pushService.getTokens(recipientId, recipientType);
        if (tokens.length > 0) {
            const title = this.templateService.render(template.pushTitle, data);
            const body = this.templateService.render(template.pushBody, data);
            await this.pushService.sendToTokens(tokens, title, body, data);
            deliveryStatus.push = { status: 'sent', sentAt: new Date() };
        }
        break;
    }
} catch (error) {
    deliveryStatus[channel] = {
        status: 'failed',
        error: error.message,
        failedAt: new Date()
    };
}
});

await Promise.allSettled(promises);

// Save notification record
await this.notificationRepository.save({
    userId: recipientType === 'user' ? recipientId : null,
    partnerId: recipientType === 'partner' ? recipientId : null,
    notificationType: type,
    title: this.templateService.render(template.pushTitle || template.emailSubject, data),
    message: this.templateService.render(template.pushBody || template.smsTemplate, data),
    channels,
    deliveryStatus,
    metadata: data
});
}

// Notification event handlers
async onBookingCreated(booking: Booking) {
    await this.send({
        recipientId: booking.userId,
        recipientType: 'user',
        type: 'booking_created',
        channels: ['sms', 'whatsapp', 'email'],
        data: {

```

```
        bookingNumber: booking.bookingNumber,
        serviceName: booking.service.name,
        bookingDate: booking.bookingDate,
        timeSlot: booking.timeSlot
    }
});
}
```

```
async onPartnerAssigned(booking: Booking, partner: Partner) {
```

```
    // Notify customer
```

```
    await this.send({
        recipientId: booking.userId,
        recipientType: 'user',
        type: 'partner_assigned',
        channels: ['sms', 'whatsapp', 'push'],
        data: {
            bookingNumber: booking.bookingNumber,
            partnerName: partner.fullName,
            partnerPhone: partner.phone,
            partnerRating: partner.averageRating
        }
    });
```

```
    // Notify partner
```

```
    await this.send({
        recipientId: partner.id,
        recipientType: 'partner',
        type: 'job_assigned',
        channels: ['sms', 'push'],
        data: {
            bookingNumber: booking.bookingNumber,
            customerName: booking.user?.fullName || booking.guestName,
            serviceName: booking.service.name,
            address: booking.address.streetAddress,
            timeSlot: booking.timeSlot
        }
    });
}
```

```
async onServiceCompleted(booking: Booking) {
```

```
    await this.send({
        recipientId: booking.userId,
        recipientType: 'user',
        type: 'service_completed',
        channels: ['whatsapp', 'email', 'push'],
        data: {
            bookingNumber: booking.bookingNumber,
```



```

        serviceName: booking.service.name,
        amount: booking.totalAmount,
        partnerName: booking.partner.fullName
    }
});
}

async onPayoutProcessed(payout: PartnerPayout) {
    await this.send({
        recipientId: payout.partnerId,
        recipientType: 'partner',
        type: 'payout_processed',
        channels: ['sms', 'email'],
        data: {
            amount: payout.netPayoutAmount,
            periodStart: payout.periodStartDate,
            periodEnd: payout.periodEndDate,
            totalBookings: payout.totalBookings,
            transactionReference: payout.transactionReference
        }
    });
}
}

```

8.2 SMS Integration

typescript

```
// SMS Service (MSG91 / Twilio)
class SmsService {
  async send(phone: string, message: string) {
    try {
      // MSG91 API
      const response = await axios.post(
        'https://api.msg91.com/api/v5/flow/',
        {
          template_id: process.env.MSG91_TEMPLATE_ID,
          short_url: '0',
          recipients: [{
            mobiles: phone,
            var1: message
          }]
        },
        {
          headers: {
            'authkey': process.env.MSG91_AUTH_KEY,
            'content-type': 'application/json'
          }
        }
      );

      return response.data;
    } catch (error) {
      console.error('SMS send failed:', error);
      throw error;
    }
  }
}
```

8.3 WhatsApp Integration

typescript

```
// WhatsApp Service (Gupshup)
class WhatsappService {
  async send(phone: string, message: string) {
    try {
      const response = await axios.post(
        'https://api.gupshup.io/sm/api/v1/msg',
        new URLSearchParams({
          channel: 'whatsapp',
          source: process.env.WHATSAPP_SOURCE_NUMBER,
          destination: phone,
          'src.name': 'Groomsta',
          message: JSON.stringify({
            type: 'text',
            text: message
          })
        }),
        {
          headers: {
            'apikey': process.env.GUPSHUP_API_KEY,
            'Content-Type': 'application/x-www-form-urlencoded'
          }
        }
      );

      return response.data;
    } catch (error) {
      console.error('WhatsApp send failed:', error);
      throw error;
    }
  }
}
```

8.4 Push Notifications

typescript

```
// Firebase Cloud Messaging Service
```

```
class PushNotificationService {  
  private admin: admin.app.App;  
  
  constructor() {  
    this.admin = admin.initializeApp({  
      credential: admin.credential.cert({  
        projectId: process.env.FIREBASE_PROJECT_ID,  
        clientEmail: process.env.FIREBASE_CLIENT_EMAIL,  
        privateKey: process.env.FIREBASE_PRIVATE_KEY.replace(/\n/g, '\n')  
      })  
    });  
  }  
}
```

```
  async sendToTokens(  
    tokens: string[],  
    title: string,  
    body: string,  
    data?: any  
  ) {  
    const message = {  
      notification: {  
        title,  
        body  
      },  
      data: data || {},  
      tokens  
    };  
  
    const response = await this.admin.messaging().sendMulticast(message);
```

```
    // Remove invalid tokens
```

```
    if (response.failureCount > 0) {  
      const failedTokens = [];  
      response.responses.forEach((resp, idx) => {  
        if (!resp.success) {  
          failedTokens.push(tokens[idx]);  
        }  
      });  
      await this.removeInvalidTokens(failedTokens);  
    }  
  
    return response;  
  }
```

```
  async getTokens(userId: string, userType: string) {
```

```
const tokens = await this.tokenRepository.find({
  where: {
    ...(userType === 'user' ? { userId } : { partnerId: userId }),
    isActive: true
  }
});

return tokens.map(t => t.token);
}

async removeInvalidTokens(tokens: string[]) {
  await this.tokenRepository.update(
    { token: In(tokens) },
    { isActive: false }
  );
}
}
```

9. Cron Jobs & Scheduled Tasks

9.1 Task Scheduler Setup

typescript

```
// Using Bull Queue for job scheduling
```

```
import Queue from 'bull';
```

```
class TaskScheduler {
```

```
  private payoutQueue: Queue.Queue;
```

```
  private reminderQueue: Queue.Queue;
```

```
  private cleanupQueue: Queue.Queue;
```

```
  constructor() {
```

```
    this.payoutQueue = new Queue('payouts', {
```

```
      redis: {
```

```
        host: process.env.REDIS_HOST,
```

```
        port: parseInt(process.env.REDIS_PORT)
```

```
      }
```

```
    });
```

```
    this.reminderQueue = new Queue('reminders', {
```

```
      redis: {
```

```
        host: process.env.REDIS_HOST,
```

```
        port: parseInt(process.env.REDIS_PORT)
```

```
      }
```

```
    });
```

```
    this.cleanupQueue = new Queue('cleanup', {
```

```
      redis: {
```

```
        host: process.env.REDIS_HOST,
```

```
        port: parseInt(process.env.REDIS_PORT)
```

```
      }
```

```
    });
```

```
    this.setupProcessors();
```

```
  }
```

```
  private setupProcessors() {
```

```
    // Weekly payout processing
```

```
    this.payoutQueue.process('generate-weekly-payouts', async (job) => {
```

```
      return await this.generateWeeklyPayouts();
```

```
    });
```

```
    // Booking reminders
```

```
    this.reminderQueue.process('booking-reminder', async (job) => {
```

```
      return await this.sendBookingReminders(job.data);
```

```
    });
```

```
    // Cleanup expired OTPs
```

```
    this.cleanupQueue.process('cleanup-otps', async (job) => {
```

```
return await this.cleanupExpiredOTPs();
});
}
}
```

9.2 Scheduled Jobs

typescript

// Cron Jobs

`@Injectable()`

`export class CronService {`

`constructor(`

`private readonly payoutService: PayoutService,`

`private readonly bookingService: BookingService,`

`private readonly notificationService: NotificationService,`

`private readonly cleanupService: CleanupService`

`) {}`

// Every Monday at 12:00 AM - Generate weekly payouts

`@Cron('0 0 * * 1')`

`async generateWeeklyPayouts() {`

`console.log('Starting weekly payout generation...');`

`const endDate = new Date();`

`const startDate = new Date();`

`startDate.setDate(startDate.getDate() - 7);`

`await this.payoutService.generatePayoutBatch(startDate, endDate);`

`}`

// Every day at 9:00 AM - Send booking reminders

`@Cron('0 9 * * *')`

`async sendBookingReminders() {`

`const tomorrow = new Date();`

`tomorrow.setDate(tomorrow.getDate() + 1);`

`const bookings = await this.bookingService.findByDate(tomorrow);`

`for (const booking of bookings) {`

`await this.notificationService.send({`

`recipientId: booking.userId,`

`recipientType: 'user',`

`type: 'booking_reminder',`

`channels: ['sms', 'whatsapp'],`

`data: {`

`bookingNumber: booking.bookingNumber,`

`serviceName: booking.service.name,`

`date: booking.bookingDate,`

`timeSlot: booking.timeSlot`

`}`

`});`

`}`

`}`

// Every hour - Cleanup expired OTPs

```
@Cron('0 * * * *')
async cleanupExpiredOTPs() {
  await this.cleanupService.deleteExpiredOTPs();
}
```

// Every hour - Expire inactive booking requests

```
@Cron('0 * * * *')
async expireBookingRequests() {
  const expiredRequests = await this.bookingService.findExpiredRequests();

  for (const request of expiredRequests) {
    await this.bookingService.markRequestExpired(request.id);
  }
}
```

// Every day at 2:00 AM - Clean up old sessions

```
@Cron('0 2 * * *')
async cleanupOldSessions() {
  await this.cleanupService.deleteExpiredSessions();
}
```

// Every day at 3:00 AM - Generate daily reports

```
@Cron('0 3 * * *')
async generateDailyReports() {
  const yesterday = new Date();
  yesterday.setDate(yesterday.getDate() - 1);

  await this.reportService.generateDailyReport(yesterday);
}
```

// Every 15 minutes - Check for stuck bookings

```
@Cron('* /15 * * * *')
async checkStuckBookings() {
  const stuckBookings = await this.bookingService.findStuckBookings();

  for (const booking of stuckBookings) {
    // Auto-cancel or reassign
    await this.bookingService.handleStuckBooking(booking);
  }
}
```

// First day of every month - Update membership status

```
@Cron('0 0 1 * *')
async updateMembershipStatus() {
  await this.membershipService.processExpiredMemberships();
}
```

```
}  
}
```

10. File Storage

10.1 AWS S3 Integration

typescript

```
// S3 Service
```

```
import { S3 } from 'aws-sdk';
```

```
class StorageService {
```

```
  private s3: S3;
```

```
  private bucketName: string;
```

```
  constructor() {
```

```
    this.s3 = new S3({  
      accessKeyId: process.env.AWS_ACCESS_KEY_ID,  
      secretAccessKey: process.env.AWS_SECRET_ACCESS_KEY,  
      region: process.env.AWS_REGION  
    });
```

```
    this.bucketName = process.env.AWS_S3_BUCKET;
```

```
  }
```

```
  async uploadFile(  
    file: Express.Multer.File,  
    folder: string  
  ): Promise<string> {
```

```
    file: Express.Multer.File,
```

```
    folder: string
```

```
): Promise<string> {
```

```
  const fileName = `${folder}/${Date.now()}-${file.originalname}`;
```

```
  const params = {
```

```
    Bucket: this.bucketName,
```

```
    Key: fileName,
```

```
    Body: file.buffer,
```

```
    ContentType: file.mimetype,
```

```
    ACL: 'public-read'
```

```
  };
```

```
  const result = await this.s3.upload(params).promise();
```

```
  return result.Location;
```

```
}
```

```
  async deleteFile(fileUrl: string): Promise<void> {
```

```
    const key = fileUrl.split('.com/')[1];
```

```
    const params = {
```

```
      Bucket: this.bucketName,
```

```
      Key: key
```

```
    };
```

```
    await this.s3.deleteObject(params).promise();
```

```
  }
```

```
  async getSignedUrl(key: string, expiresIn: number = 3600): Promise<string> {
```

```

const params = {
  Bucket: this.bucketName,
  Key: key,
  Expires: expiresIn
};

return this.s3.getSignedUrl('getObject', params);
}

// Upload multiple files
async uploadMultipleFiles(
  files: Express.Multer.File[],
  folder: string
): Promise<string[]> {
  const uploadPromises = files.map(file => this.uploadFile(file, folder));
  return await Promise.all(uploadPromises);
}
}

// File Upload Controller
@Controller('upload')
export class UploadController {
  constructor(private readonly storageService: StorageService) {}

  @Post('partner-documents')
  @UseGuards(JwtAuthGuard)
  @UseInterceptors(FileInterceptor('document'))
  async uploadPartnerDocument(
    @UploadedFile() file: Express.Multer.File,
    @Request() req,
    @Body() body: { documentType: string }
  ) {
    // Validate file type
    const allowedTypes = ['image/jpeg', 'image/png', 'application/pdf'];
    if (!allowedTypes.includes(file.mimetype)) {
      throw new BadRequestException('Invalid file type');
    }

    // Validate file size (max 5MB)
    if (file.size > 5 * 1024 * 1024) {
      throw new BadRequestException('File size exceeds 5MB');
    }

    const fileUrl = await this.storageService.uploadFile(
      file,
      `partners/${req.user.sub}/documents`
    );
  }
}

```

```

// Save document record
await this.partnerDocumentRepository.save({
  partnerId: req.user.sub,
  documentType: body.documentType,
  documentUrl: fileUrl,
  verificationStatus: 'pending'
});

return { success: true, fileUrl };
}

@Post('profile-image')
@UseGuards(JwtAuthGuard)
@UseInterceptors(FileInterceptor('image'))
async uploadProfileImage(
  @UploadedFile() file: Express.Multer.File,
  @Request() req
) {
  // Validate image
  if (!file.mimetype.startsWith('image/')) {
    throw new BadRequestException('Only images are allowed');
  }

  const fileUrl = await this.storageService.uploadFile(
    file,
    `${req.user.type}s/${req.user.sub}/profile`
  );

  // Update user/partner profile
  if (req.user.type === 'user') {
    await this.userRepository.update(req.user.sub, { profileImageUrl: fileUrl });
  } else {
    await this.partnerRepository.update(req.user.sub, { profileImageUrl: fileUrl });
  }

  return { success: true, fileUrl };
}
}

```

11. Caching Strategy

11.1 Redis Caching Implementation

// Cache Service

```
class CacheService {  
  private redis: Redis;  
  
  constructor() {  
    this.redis = new Redis({  
      host: process.env.REDIS_HOST,  
      port: parseInt(process.env.REDIS_PORT),  
      password: process.env.REDIS_PASSWORD,  
      db: 0  
    });  
  }  
  
  async get<T>(key: string): Promise<T | null> {  
    const data = await this.redis.get(key);  
    return data ? JSON.parse(data) : null;  
  }  
  
  async set(key: string, value: any, ttl?: number): Promise<void> {  
    const serialized = JSON.stringify(value);  
    if (ttl) {  
      await this.redis.setex(key, ttl, serialized);  
    } else {  
      await this.redis.set(key, serialized);  
    }  
  }  
  
  async delete(key: string): Promise<void> {  
    await this.redis.del(key);  
  }  
  
  async deletePattern(pattern: string): Promise<void> {  
    const keys = await this.redis.keys(pattern);  
    if (keys.length > 0) {  
      await this.redis.del(...keys);  
    }  
  }  
  
  async increment(key: string, amount: number = 1): Promise<number> {  
    return await this.redis.incrby(key, amount);  
  }  
  
  async setExpire(key: string, seconds: number): Promise<void> {  
    await this.redis.expire(key, seconds);  
  }  
}
```

```

// Cache Interceptor for automatic caching
@Injectable()
export class CacheInterceptor implements NestInterceptor {
  constructor(private readonly cacheService: CacheService) {}

  async intercept(context: ExecutionContext, next: CallHandler): Promise<Observable<any>> {
    const request = context.switchToHttp().getRequest();
    const cacheKey = this.generateCacheKey(request);

    // Try to get from cache
    const cachedResponse = await this.cacheService.get(cacheKey);
    if (cachedResponse) {
      return of(cachedResponse);
    }

    // Execute request and cache response
    return next.handle().pipe(
      tap(async (response) => {
        await this.cacheService.set(cacheKey, response, 300); // 5 minutes
      })
    );
  }

  private generateCacheKey(request: any): string {
    return `cache:${request.url}:${JSON.stringify(request.query)}`;
  }
}

```

11.2 Cache Strategy by Module

typescript

```
// Services Cache - TTL: 1 hour
await cacheService.set('services:all', services, 3600);
await cacheService.set(`service:${serviceId}`, service, 3600);

// Categories Cache - TTL: 1 hour
await cacheService.set('categories:all', categories, 3600);

// Partner Cache - TTL: 5 minutes
await cacheService.set(`partner:${partnerId}:services`, partnerServices, 300);
await cacheService.set(`partner:${partnerId}:availability`, availability, 300);

// User Cache - TTL: 15 minutes
await cacheService.set(`user:${userId}:profile`, userProfile, 900);
await cacheService.set(`user:${userId}:addresses`, addresses, 900);

// Booking Cache - TTL: 1 minute (frequently changing)
await cacheService.set(`booking:${bookingId}`, booking, 60);

// Location-based Cache - TTL: 10 minutes
await cacheService.set(
  `partners:nearby:${latitude}:${longitude}`,
  nearbyPartners,
  600
);

// Session Cache - TTL: 15 minutes
await cacheService.set(`session:${sessionId}`, sessionData, 900);

// Rate Limiting Cache - TTL: 1 hour
await cacheService.increment(`ratelimit:${userId}:${endpoint}`);
await cacheService.setExpire(`ratelimit:${userId}:${endpoint}`, 3600);
```

11.3 Cache Invalidation

typescript


```
// Cache Invalidation Service
```

```
class CacheInvalidationService {  
  constructor(private readonly cacheService: CacheService) {}  
  
  async invalidateService(serviceId: string) {  
    await this.cacheService.delete(`service:${serviceId}`);  
    await this.cacheService.deletePattern('services:*');  
    await this.cacheService.deletePattern('categories:*');  
  }  
  
  async invalidatePartner(partnerId: string) {  
    await this.cacheService.deletePattern(`partner:${partnerId}:*`);  
    await this.cacheService.deletePattern('partners:nearby:*');  
  }  
  
  async invalidateUser(userId: string) {  
    await this.cacheService.deletePattern(`user:${userId}:*`);  
  }  
  
  async invalidateBooking(bookingId: string) {  
    await this.cacheService.delete(`booking:${bookingId}`);  
  }  
}
```

12. Security Considerations

12.1 Rate Limiting

```
typescript
```

```

// Rate Limiter Middleware
@Injectable()
export class RateLimiterMiddleware implements NestMiddleware {
  constructor(private readonly redis: Redis) {}

  async use(req: Request, res: Response, next: NextFunction) {
    const key = `ratelimit:${req.ip}:${req.path}`;
    const limit = this.getLimit(req.path);
    const window = 60; // 1 minute

    const current = await this.redis.incr(key);

    if (current === 1) {
      await this.redis.expire(key, window);
    }

    if (current > limit) {
      throw new HttpException('Too many requests', 429);
    }

    next();
  }

  private getLimit(path: string): number {
    // Different limits for different endpoints
    if (path.includes('/auth/send-otp')) return 3;
    if (path.includes('/bookings')) return 20;
    return 100; // Default
  }
}

// Apply rate limiting
@UseGuards(RateLimiterGuard)
@Post('/auth/send-otp')
async sendOtp(@Body() body: SendOtpDto) {
  // ...
}

```

12.2 Input Validation & Sanitization

typescript

// DTOs with validation

```
import { IsString, IsEmail, IsPhoneNumber, IsUUID, Min, Max } from 'class-validator';
import { Transform } from 'class-transformer';
import * as sanitizeHtml from 'sanitize-html';
```

```
export class CreateBookingDto {
  @IsUUID()
  serviceId: string;

  @IsString()
  @IsIn(['home', 'salon'])
  serviceType: string;

  @IsUUID()
  @IsOptional()
  addressId?: string;

  @IsDateString()
  bookingDate: string;

  @IsString()
  @IsIn(['morning', 'afternoon', 'evening'])
  timeSlot: string;

  @IsArray()
  @IsUUID('4', { each: true })
  @IsOptional()
  addons?: string[];

  @IsString()
  @MaxLength(500)
  @Transform(({ value }) => sanitizeHtml(value))
  @IsOptional()
  customerNotes?: string;
}
```

// Global Validation Pipe

```
app.useGlobalPipes(
  new ValidationPipe({
    whitelist: true, // Strip non-whitelisted properties
    forbidNonWhitelisted: true, // Throw error for non-whitelisted
    transform: true, // Auto-transform to DTO instances
    transformOptions: {
      enableImplicitConversion: true
    }
  })
)
```

```
    })  
  );
```

12.3 SQL Injection Prevention

typescript

// Using TypeORM parameterized queries (automatically prevents SQL injection)

```
const users = await this.userRepository  
  .createQueryBuilder('user')  
  .where('user.phone = :phone', { phone: phoneNumber })  
  .getMany();
```

// Never use raw queries with string concatenation

// ❌ Bad:

```
const query = `SELECT * FROM users WHERE phone = '${phoneNumber}'`;
```

// ✅ Good:

```
const query = this.userRepository  
  .query('SELECT * FROM users WHERE phone = $1', [phoneNumber]);
```

12.4 XSS Protection

typescript

// Helmet middleware for security headers

```
import helmet from 'helmet';
```

```
app.use(helmet({  
  contentSecurityPolicy: {  
    directives: {  
      defaultSrc: ["'self'"],  
      styleSrc: ["'self'", "unsafe-inline"],  
      scriptSrc: ["'self'"],  
      imgSrc: ["'self'", 'data:', 'https:'],  
    },  
  },  
  hsts: {  
    maxAge: 31536000,  
    includeSubDomains: true,  
    preload: true  
  }  
}));
```

// Sanitize all text inputs

```
import * as sanitizeHtml from 'sanitize-html';
```

```
@Transform(({ value }) => sanitizeHtml(value, {  
  allowedTags: [],  
  allowedAttributes: {}  
}))  
@IsString()  
description: string;
```

12.5 CORS Configuration

typescript

// CORS setup

```
app.enableCors({  
  origin: [  
    'https://groomsta.com',  
    'https://www.groomsta.com',  
    'https://admin.groomsta.com',  
    'http://localhost:3000' // Development only  
  ],  
  credentials: true,  
  methods: ['GET', 'POST', 'PUT', 'DELETE', 'PATCH'],  
  allowedHeaders: ['Content-Type', 'Authorization']  
});
```

12.6 Secrets Management

typescript

```
// Environment variables structure
```

```
// .env file (never commit to git)
```

```
DATABASE_URL=postgresql://user:pass@localhost:5432/groomsta
```

```
REDIS_HOST=localhost
```

```
REDIS_PORT=6379
```

```
REDIS_PASSWORD=secret
```

```
JWT_SECRET=your-very-long-random-secret-key
```

```
JWT_REFRESH_SECRET=another-very-long-random-secret-key
```

```
RAZORPAY_KEY_ID=rzp_test_xxx
```

```
RAZORPAY_KEY_SECRET=xxx
```

```
RAZORPAY_WEBHOOK_SECRET=xxx
```

```
AWS_ACCESS_KEY_ID=xxx
```

```
AWS_SECRET_ACCESS_KEY=xxx
```

```
AWS_REGION=ap-south-1
```

```
AWS_S3_BUCKET=groomsta-uploads
```

```
MSG91_AUTH_KEY=xxx
```

```
GUPSHUP_API_KEY=xxx
```

```
FIREBASE_PROJECT_ID=xxx
```

```
FIREBASE_PRIVATE_KEY=xxx
```

```
// Config service
```

```
@Injectable()
```

```
export class ConfigService {
```

```
  get(key: string): string {
```

```
    const value = process.env[key];
```

```
    if (!value) {
```

```
      throw new Error(`Config key ${key} is missing`);
```

```
    }
```

```
    return value;
```

```
  }
```

```
  getDatabaseConfig() {
```

```
    return {
```

```
      type: 'postgres',
```

```
      url: this.get('DATABASE_URL'),
```

```
      entities: [__dirname + '/../**/*.entity{.ts,.js}'],
```

```
      synchronize: false, // Always false in production
```

```
      logging: process.env.NODE_ENV === 'development',
```

```
      ssl: process.env.NODE_ENV === 'production' ? { rejectUnauthorized: false } : false
```

```
    };
```

13. Error Handling

13.1 Global Exception Filter

typescript

// Global Exception Filter

`@Catch()`

`export class AllExceptionsFilter implements ExceptionFilter {`

`constructor(private readonly logger: Logger) {}`

`catch(exception: unknown, host: ArgumentsHost) {`

`const ctx = host.switchToHttp();`

`const response = ctx.getResponse<Response>();`

`const request = ctx.getRequest<Request>();`

`let status = 500;`

`let message = 'Internal server error';`

`let code = 'INTERNAL__ERROR';`

`let details = null;`

`if (exception instanceof HttpException) {`

`status = exception.getStatus();`

`const exceptionResponse = exception.getResponse();`

`if (typeof exceptionResponse === 'object') {`

`message = exceptionResponse['message'] || message;`

`details = exceptionResponse['details'] || null;`

`} else {`

`message = exceptionResponse;`

`}`

`code = this.getErrorCode(status);`

`} else if (exception instanceof Error) {`

`message = exception.message;`

`this.logger.error(exception.stack);`

`}`

// Log error

`this.logger.error(`

``${request.method} ${request.url} - ${status} - ${message}`,`

`exception instanceof Error ? exception.stack : "`

`);`

// Send response

`response.status(status).json({`

`success: false,`

`error: {`

`code,`

`message,`

`details`

`},`

```
metadata: {
  timestamp: new Date().toISOString(),
  path: request.url,
  method: request.method,
  requestId: request['id']
}
});
}

private getErrorCode(status: number): string {
  const codes = {
    400: 'BAD_REQUEST',
    401: 'UNAUTHORIZED',
    403: 'FORBIDDEN',
    404: 'NOT_FOUND',
    409: 'CONFLICT',
    422: 'VALIDATION_ERROR',
    429: 'RATE_LIMIT_EXCEEDED',
    500: 'INTERNAL_ERROR',
    503: 'SERVICE_UNAVAILABLE'
  };
  return codes[status] || 'UNKNOWN_ERROR';
}
}
```

13.2 Custom Exception Classes

typescript

// Custom Exceptions

```
export class BusinessException extends HttpException {
  constructor(message: string, code: string = 'BUSINESS_ERROR') {
    super({ message, code }, 400);
  }
}

export class ResourceNotFoundException extends HttpException {
  constructor(resource: string, id: string) {
    super({
      message: `${resource} with id ${id} not found`,
      code: 'RESOURCE_NOT_FOUND'
    }, 404);
  }
}

export class InsufficientBalanceException extends BusinessException {
  constructor() {
    super('Insufficient wallet balance', 'INSUFFICIENT_BALANCE');
  }
}

export class BookingNotFoundException extends ResourceNotFoundException {
  constructor(bookingId: string) {
    super('Booking', bookingId);
  }
}

// Usage
if (!booking) {
  throw new BookingNotFoundException(bookingId);
}

if (walletBalance < amount) {
  throw new InsufficientBalanceException();
}
```

14. Logging & Monitoring

14.1 Winston Logger Setup

typescript

// Logger Configuration

```
import * as winston from 'winston';  
import * as DailyRotateFile from 'winston-daily-rotate-file';
```

```
const logger = winston.createLogger({  
  level: process.env.LOG_LEVEL || 'info',  
  format: winston.format.combine(  
    winston.format.timestamp(),  
    winston.format.errors({ stack: true }),  
    winston.format.json()  
  ),  
  defaultMeta: {  
    service: 'groomsta-api',  
    environment: process.env.NODE_ENV  
  },  
  transports: [  
    // Error logs  
    new DailyRotateFile({  
      filename: 'logs/error-%DATE%.log',  
      datePattern: 'YYYY-MM-DD',  
      level: 'error',  
      maxFiles: '30d',  
      maxSize: '20m'  
    }),  
  
    // Combined logs  
    new DailyRotateFile({  
      filename: 'logs/combined-%DATE%.log',  
      datePattern: 'YYYY-MM-DD',  
      maxFiles: '14d',  
      maxSize: '20m'  
    }),  
  
    // Console output (development)  
    new winston.transports.Console({  
      format: winston.format.combine(  
        winston.format.colorize(),  
        winston.format.simple()  
      )  
    })  
  ]  
});
```

// Logger Service

```
@Injectable()  
export class LoggerService {
```

```

private logger: winston.Logger;

constructor() {
  this.logger = logger;
}

log(message: string, context?: string) {
  this.logger.info(message, { context });
}

error(message: string, trace?: string, context?: string) {
  this.logger.error(message, { trace, context });
}

warn(message: string, context?: string) {
  this.logger.warn(message, { context });
}

debug(message: string, context?: string) {
  this.logger.debug(message, { context });
}

// Structured logging
logBookingCreated(booking: any) {
  this.logger.info('Booking created', {
    event: 'booking_created',
    bookingId: booking.id,
    userId: booking.userId,
    serviceId: booking.serviceId,
    amount: booking.totalAmount
  });
}

logPaymentProcessed(payment: any) {
  this.logger.info('Payment processed', {
    event: 'payment_processed',
    paymentId: payment.id,
    amount: payment.amount,
    status: payment.paymentStatus
  });
}

```

14.2 Application Performance Monitoring

typescript

// Sentry Integration for Error Tracking

```
import * as Sentry from '@sentry/node';
```

```
Sentry.init({  
  dsn: process.env.SENTRY_DSN,  
  environment: process.env.NODE_ENV,  
  tracesSampleRate: 1.0  
});
```

// Sentry Interceptor

```
@Injectable()
```

```
export class SentryInterceptor implements NestInterceptor {  
  intercept(context: ExecutionContext, next: CallHandler): Observable<any> {  
    return next.handle().pipe(  
      catchError((error) => {  
        Sentry.captureException(error);  
        throw error;  
      })  
    );  
  }  
}
```

// Performance monitoring interceptor

```
@Injectable()
```

```
export class PerformanceInterceptor implements NestInterceptor {  
  constructor(private readonly logger: LoggerService) {}  
  
  intercept(context: ExecutionContext, next: CallHandler): Observable<any> {  
    const request = context.switchToHttp().getRequest();  
    const { method, url } = request;  
    const start = Date.now();  
  
    return next.handle().pipe(  
      tap(() => {  
        const duration = Date.now() - start;  
        this.logger.log(`${method} ${url} - ${duration}ms`, 'Performance');  
  
        // Alert if slow  
        if (duration > 1000) {  
          this.logger.warn('Slow request: ${method} ${url} - ${duration}ms', 'Performance');  
        }  
      })  
    );  
  }  
}
```

14.3 Health Checks

typescript

```
// Health Check Controller
```

```
@Controller('health')
```

```
export class HealthController {
```

```
  constructor(
```

```
    private readonly db: DataSource,
```

```
    private readonly redis: Redis
```

```
) {}
```

```
@Get()
```

```
async check() {
```

```
  const health = {
```

```
    status: 'ok',
```

```
    timestamp: new Date().toISOString(),
```

```
    uptime: process.uptime(),
```

```
    checks: {
```

```
      database: await this.checkDatabase(),
```

```
      redis: await this.checkRedis(),
```

```
      memory: this.checkMemory()
```

```
    }
```

```
  };
```

```
const allHealthy = Object.values(health.checks).every(check => check.status === 'ok');
```

```
const status = allHealthy ? 200 : 503;
```

```
return { ...health, status: allHealthy ? 'ok' : 'degraded' };
```

```
}
```

```
private async checkDatabase() {
```

```
  try {
```

```
    await this.db.query('SELECT 1');
```

```
    return { status: 'ok', message: 'Database is reachable' };
```

```
  } catch (error) {
```

```
    return { status: 'error', message: error.message };
```

```
  }
```

```
}
```

```
private async checkRedis() {
```

```
  try {
```

```
    await this.redis.ping();
```

```
    return { status: 'ok', message: 'Redis is reachable' };
```

```
  } catch (error) {
```

```
    return { status: 'error', message: error.message };
```

```
  }
```

```
}
```

```
private checkMemory() {
```



```
const used = process.memoryUsage();  
return {  
  status: 'ok',  
  heapUsed: `${Math.round(used.heapUsed / 1024 / 1024)} MB`,  
  heapTotal: `${Math.round(used.heapTotal / 1024 / 1024)} MB`,  
  rss: `${Math.round(used.rss / 1024 / 1024)} MB`  
};  
}  
}
```

15. Deployment Architecture

15.1 Docker Configuration

dockerfile

Dockerfile

FROM node:20-alpine **AS** builder

WORKDIR /app

Copy package files

COPY package*.json ./

RUN npm ci --only=production && npm cache clean --force

Copy source code

COPY . .

Build application

RUN npm run build

Production image

FROM node:20-alpine

WORKDIR /app

Copy built application

COPY --from=builder /app/dist ./dist

COPY --from=builder /app/node_modules ./node_modules

COPY --from=builder /app/package*.json ./

Set environment

ENV NODE_ENV=production

Expose port

EXPOSE 3000

Start application

CMD ["node", "dist/main.js"]

yaml

```
# docker-compose.yml
```

```
version: '3.8'
```

```
services:
```

```
  api:
```

```
    build: .
```

```
    ports:
```

```
      - "3000:3000"
```

```
    environment:
```

```
      - NODE_ENV=production
```

```
      - DATABASE_URL=postgresql://postgres:password@postgres:5432/groomsta
```

```
      - REDIS_HOST=redis
```

```
    depends_on:
```

```
      - postgres
```

```
      - redis
```

```
    restart: unless-stopped
```

```
  postgres:
```

```
    image: postgres:16-alpine
```

```
    volumes:
```

```
      - postgres_data:/var/lib/postgresql/data
```

```
    environment:
```

```
      - POSTGRES_DB=groomsta
```

```
      - POSTGRES_USER=postgres
```

```
      - POSTGRES_PASSWORD=password
```

```
    ports:
```

```
      - "5432:5432"
```

```
    restart: unless-stopped
```

```
  redis:
```

```
    image: redis:7-alpine
```

```
    volumes:
```

```
      - redis_data:/data
```

```
    ports:
```

```
      - "6379:6379"
```

```
    restart: unless-stopped
```

```
  nginx:
```

```
    image: nginx:alpine
```

```
    ports:
```

```
      - "80:80"
```

```
      - "443:443"
```

```
    volumes:
```

```
      - ./nginx.conf:/etc/nginx/nginx.conf
```

```
      - ./ssl:/etc/nginx/ssl
```

```
    depends_on:
```

```
- api
```

```
restart: unless-stopped
```

```
volumes:
```

```
  postgres_data:
```

```
  redis_data:
```

15.2 CI/CD Pipeline

```
yaml
```

```
# .github/workflows/deploy.yml
```

```
name: Deploy to Production
```

```
on:
```

```
  push:
```

```
    branches: [main]
```

```
jobs:
```

```
  test:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - uses: actions/checkout@v3
```

```
      - name: Setup Node.js
```

```
        uses: actions/setup-node@v3
```

```
        with:
```

```
          node-version: '20'
```

```
      - name: Install dependencies
```

```
        run: npm ci
```

```
      - name: Run tests
```

```
        run: npm test
```

```
      - name: Run linter
```

```
        run: npm run lint
```

```
  build:
```

```
    needs: test
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - uses: actions/checkout@v3
```

```
      - name: Build Docker image
```

```
        run: docker build -t groomsta-api:${{ github.sha }} .
```

```
      - name: Push to Registry
```

```
        run: |
```

```
          docker tag groomsta-api:${{ github.sha }} registry.example.com/groomsta-api:latest
```

```
          docker push registry.example.com/groomsta-api:latest
```

```
  deploy:
```

```
    needs: build
```

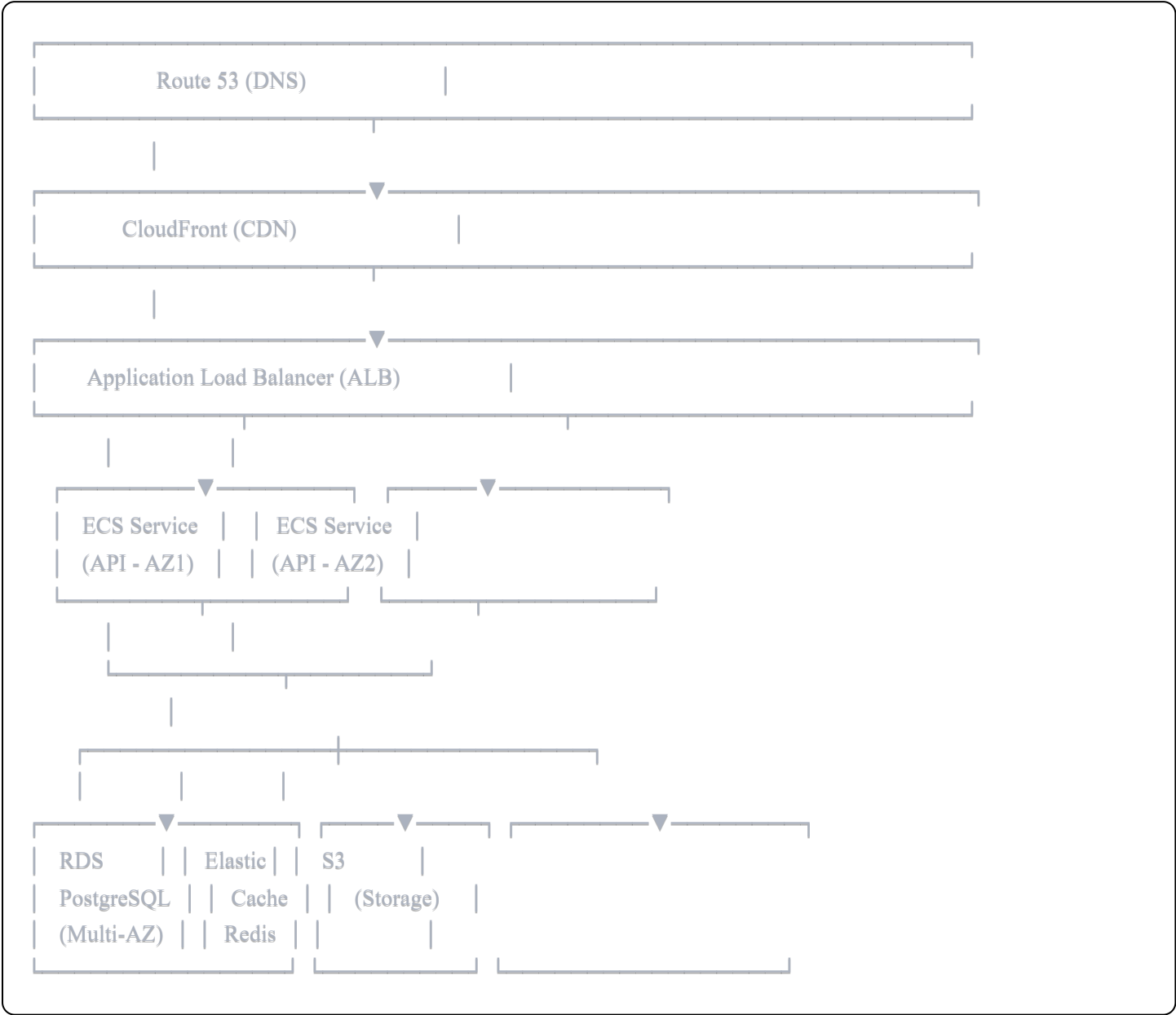
```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - name: Deploy to AWS ECS
```

```
run: |
aws ecs update-service \
  --cluster groomsta-cluster \
  --service groomsta-api \
  --force-new-deployment
```

15.3 Production Environment



Document Summary

This backend structure document provides a complete foundation for building Groomsta's backend system with:

- ✓ **Scalable Architecture** - Modular design ready for microservices migration
- ✓ **Comprehensive Database Schema** - 30+ tables with relationships and constraints
- ✓ **RESTful API Design** - 100+ endpoints across all modules
- ✓ **Robust Authentication** - JWT + OTP with role-based access control
- ✓ **Real-time Communication** - WebSocket for live updates

- ✓ **Payment Integration** - Razorpay with wallet system
- ✓ **Multi-channel Notifications** - SMS, Email, WhatsApp, Push
- ✓ **Automated Tasks** - Cron jobs for payouts and reminders
- ✓ **Security Best Practices** - Rate limiting, validation, encryption
- ✓ **Performance Optimization** - Redis caching strategy
- ✓ **Production-Ready** - Docker, CI/CD, monitoring, logging

Next Steps:

1. Set up development environment
2. Initialize NestJS project structure
3. Implement database migrations
4. Build authentication module
5. Develop core booking flow
6. Integrate payment gateway
7. Implement notification system
8. Deploy to staging environment

Estimated Development Timeline: 12-16 weeks for MVP, 2) NOT NULL,

```

-- Status
status VARCHAR(50) NOT NULL DEFAULT 'pending',
-- 'pending', 'partner_assigned', 'confirmed', 'on_the_way',
-- 'in_progress', 'completed', 'cancelled', 'refunded'

-- Timestamps
partner_assigned_at TIMESTAMP,
confirmed_at TIMESTAMP,
started_at TIMESTAMP,
completed_at TIMESTAMP,
cancelled_at TIMESTAMP,

-- Cancellation
cancelled_by VARCHAR(50), -- 'customer', 'partner', 'admin'
cancellation_reason TEXT,

-- Notes
customer_notes TEXT,
partner_notes TEXT,
admin_notes TEXT,

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

INDEX idx_bookings_user (user_id),
INDEX idx_bookings_partner (partner_id),
INDEX idx_bookings_date (booking_date),
INDEX idx_bookings_status (status),
INDEX idx_bookings_number (booking_number)

```

```
);
```

```
-- Booking Add-ons
```

```

CREATE TABLE booking_addons (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
booking_id UUID REFERENCES bookings(id) ON DELETE CASCADE,
addon_id UUID REFERENCES service_addons(id),
addon_name VARCHAR(255) NOT NULL,
addon_price DECIMAL(10, 2) NOT NULL,
quantity INT DEFAULT 1,

```

```
INDEX idx_booking_addons (booking_id)
```

```
);
```


-- Booking Requests (Job Assignment System)

```
CREATE TABLE booking_requests (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  booking_id UUID REFERENCES bookings(id) ON DELETE CASCADE,  
  partner_id UUID REFERENCES partners(id) ON DELETE CASCADE,
```

-- Request Details

```
  request_status VARCHAR(50) NOT NULL DEFAULT 'sent',  
  -- 'sent', 'seen', 'accepted', 'rejected', 'expired'
```

```
  sent_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  seen_at TIMESTAMP,  
  responded_at TIMESTAMP,  
  expires_at TIMESTAMP NOT NULL, -- 20 seconds from sent_at
```

-- Partner offer

```
  offered_price DECIMAL(10, 2),
```

-- Metadata

```
  distance_km DECIMAL(5, 2),  
  estimated_travel_time_minutes INT,
```

```
  INDEX idx_booking_requests_booking (booking_id),  
  INDEX idx_booking_requests_partner (partner_id),  
  INDEX idx_booking_requests_status (request_status)
```

);

-- Booking Status History

```
CREATE TABLE booking_status_history (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  booking_id UUID REFERENCES bookings(id) ON DELETE CASCADE,  
  from_status VARCHAR(50),  
  to_status VARCHAR(50) NOT NULL,  
  changed_by_user_id UUID REFERENCES users(id),  
  changed_by_partner_id UUID REFERENCES partners(id),  
  changed_by_admin_id UUID REFERENCES admin_users(id),  
  notes TEXT,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
  INDEX idx_status_history_booking (booking_id)
```

);

3.1.6 Payments & Wallet

```sql

-- Payments

CREATE TABLE payments (

id UUID PRIMARY KEY DEFAULT gen\_random\_uuid(),

booking\_id UUID REFERENCES bookings(id),

user\_id UUID REFERENCES users(id),

-- Payment Details

payment\_type VARCHAR(50) NOT NULL, -- 'full', 'partial', 'post\_service'

payment\_method VARCHAR(50) NOT NULL, -- 'online', 'cash', 'wallet'

payment\_stage VARCHAR(50) NOT NULL, -- 'advance', 'final'

-- Amounts

amount DECIMAL(10, 2) NOT NULL,

wallet\_amount\_used DECIMAL(10, 2) DEFAULT 0.0,

gateway\_amount DECIMAL(10, 2) DEFAULT 0.0,

-- Gateway Info

payment\_gateway VARCHAR(50), -- 'razorpay', 'cash'

gateway\_transaction\_id VARCHAR(255),

gateway\_order\_id VARCHAR(255),

gateway\_payment\_id VARCHAR(255),

gateway\_response JSONB,

-- Status

payment\_status VARCHAR(50) NOT NULL DEFAULT 'pending',

-- 'pending', 'processing', 'success', 'failed', 'refunded'

-- Timestamps

initiated\_at TIMESTAMP DEFAULT CURRENT\_TIMESTAMP,

completed\_at TIMESTAMP,

failed\_at TIMESTAMP,

failure\_reason TEXT,

created\_at TIMESTAMP DEFAULT CURRENT\_TIMESTAMP,

updated\_at TIMESTAMP DEFAULT CURRENT\_TIMESTAMP,

INDEX idx\_payments\_booking (booking\_id),

INDEX idx\_payments\_user (user\_id),

INDEX idx\_payments\_status (payment\_status),

INDEX idx\_payments\_gateway\_order (gateway\_order\_id)

);

-- User Wallet

```
CREATE TABLE user_wallets (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 user_id UUID UNIQUE REFERENCES users(id) ON DELETE CASCADE,
 balance DECIMAL(10, 2) DEFAULT 0.0,
 total_credited DECIMAL(10, 2) DEFAULT 0.0,
 total_debited DECIMAL(10, 2) DEFAULT 0.0,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

 INDEX idx_wallet_user (user_id),
 CHECK (balance >= 0)
);
```

-- Wallet Transactions

```
CREATE TABLE wallet_transactions (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 wallet_id UUID REFERENCES user_wallets(id) ON DELETE CASCADE,
 user_id UUID REFERENCES users(id),
```

-- Transaction Details

```
transaction_type VARCHAR(50) NOT NULL,
-- 'credit', 'debit', 'refund', 'referral', 'membership_discount'
```

```
amount DECIMAL(10, 2) NOT NULL,
balance_before DECIMAL(10, 2) NOT NULL,
balance_after DECIMAL(10, 2) NOT NULL,
```

-- References

```
booking_id UUID REFERENCES bookings(id),
payment_id UUID REFERENCES payments(id),
referral_id UUID REFERENCES referrals(id),
```

```
description TEXT,
metadata JSONB,
```

```
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
INDEX idx_wallet_txn_wallet (wallet_id),
INDEX idx_wallet_txn_user (user_id),
INDEX idx_wallet_txn_type (transaction_type),
INDEX idx_wallet_txn_created (created_at DESC)
```

);

-- Refunds

```
CREATE TABLE refunds (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
payment_id UUID REFERENCES payments(id),
booking_id UUID REFERENCES bookings(id),
user_id UUID REFERENCES users(id),

refund_amount DECIMAL(10, 2) NOT NULL,
refund_type VARCHAR(50) NOT NULL, -- 'full', 'partial'
refund_method VARCHAR(50) NOT NULL, -- 'wallet', 'original_source'
```

```
-- Gateway Info
```

```
gateway_refund_id VARCHAR(255),
gateway_response JSONB,
```

```
refund_status VARCHAR(50) NOT NULL DEFAULT 'pending',
-- 'pending', 'processing', 'completed', 'failed'
```

```
reason TEXT,
initiated_by VARCHAR(50), -- 'customer', 'partner', 'admin', 'system'
```

```
initiated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
completed_at TIMESTAMP,
```

```
INDEX idx_refunds_payment (payment_id),
INDEX idx_refunds_booking (booking_id),
INDEX idx_refunds_user (user_id)
```

```
);
``
```

#### #### 3.1.7 Partner Payouts

```
``sql
```

```
-- Payout Batches
```

```
CREATE TABLE payout_batches (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 batch_number VARCHAR(50) UNIQUE NOT NULL, -- PAY-2024-W01
```

```
-- Period
```

```
period_start_date DATE NOT NULL,
period_end_date DATE NOT NULL,
```

```
-- Amounts
```

```
total_partners INT NOT NULL,
total_gross_amount DECIMAL(12, 2) NOT NULL,
total_commission DECIMAL(12, 2) NOT NULL,
total_net_amount DECIMAL(12, 2) NOT NULL,
```

```
-- Status
```

```
payout_status VARCHAR(50) NOT NULL DEFAULT 'pending',
-- 'pending', 'processing', 'completed', 'failed', 'partially_completed'
```

-- Processing

generated\_at TIMESTAMP DEFAULT CURRENT\_TIMESTAMP,  
processed\_at TIMESTAMP,  
processed\_by UUID REFERENCES admin\_users(id),

notes TEXT,

INDEX idx\_payout\_batches\_status (payout\_status),

INDEX idx\_payout\_batches\_period (period\_start\_date, period\_end\_date)

);

-- Individual Payouts

CREATE TABLE partner\_payouts (

id UUID PRIMARY KEY DEFAULT gen\_random\_uuid(),

payout\_batch\_id UUID REFERENCES payout\_batches(id),

partner\_id UUID REFERENCES partners(id),

-- Period

period\_start\_date DATE NOT NULL,

period\_end\_date DATE NOT NULL,

-- Amounts

total\_bookings INT NOT NULL,

gross\_earnings DECIMAL(10, 2) NOT NULL,

commission\_amount DECIMAL(10, 2) NOT NULL,

commission\_percent DECIMAL(5